

Dissertation Title
on

**PRIVACY USING CONTEXT BASED ACCESS CONTROL
AND DATABASE SECURITY IN ANDROID DEVICES**

Submitted
in fulfillment of
the requirement for the Degree of Master of Technology

M.Tech(Network Infrastructure Management System)

By

Charuli S. Patil
(182171006)

Under the Guidance of
Prof K. K. Joshi



DEPARTMENT OF COMPUTER ENGINEERING
VEERMATA JIJABAI TECHNOLOGICAL INSTITUTE
(An Autonomous Institute Affiliated to Mumbai University)
(Central Technological Institute, Maharashtra State)
Matunga, MUMBAI - 400019

CERTIFICATE

This is to certify that **Charuli Patil [182171006]**, a student of **M.Tech (NIMS)**, has successfully completed dissertation entitled, **“Privacy using Context Based Access Control and Database Security in Android Devices”** to our satisfaction.

Project Supervisor

Head of Department

Prof K. K. Joshi

Department of Computer Engineering
and Information Technology

Dr. M.M. Chandane

Department of Computer Engineering
and Information Technology

Place: V.J.T.I, Mumbai.

Date: / /2020

APPROVAL SHEET

The dissertation titled “*Privacy using Context Based Access Control and Database Security in Android Devices*” submitted by **Charuli Patil [182171006]** is found to be satisfactory and is approved for the Degree of **Master of Technology**.

Prof K. K. Joshi
(*Project Supervisor*)

Examiner

Place: V.J.T.I, Mumbai.

Date: / /2020

STATEMENT OF CANDIDATE

I state that work embodied in this Project entitled “**Privacy using Context Based Access Control and Database Security in Android Devices**” form my own contribution of work under the guidance of **Prof K. K. Joshi** at the Department of Computer Engineering, Veermata Jijabai Technological Institute, Mumbai. The report reflects the work done during the period of candidature but may include related preliminary material provided that it has not contributed to an award of previous degree. No part of this work has been used by us for the requirement of another degree except where explicitly stated in the body of the text and the attached statement.

Charuli Patil
Roll No.- 182171006
Date: / /2020
Place: V.J.T.I Mumbai

ACKNOWLEDGEMENT

My sincere thanks go to our Director **Dr. Dhiren Patel**, Director of Veermata Jijabai Technological Institute, Mumbai, who provided me with an opportunity to work as a research scholar and gave me access to the laboratory and facilities which helped me to conduct this research.

I would like to thank all those people whose support and cooperation has been an invaluable asset during the course of this project. I would also like to thank our guide **Prof K. K. Joshi** for guiding me throughout this project and giving it the present shape. With his encouraging words and reminders, he empowered me to improve in personal as well as academic life. It would have been impossible to complete this project without his support, valuable suggestions, criticism, encouragement and guidance.

I convey my gratitude also to **Dr.M.M.Chandane**, Head of Department for his motivation and providing various facilities, which helped us in the process of this whole project.

I am also grateful to all the teaching and non-teaching staff of the Computer Engineering and Information Technology Department for directly or indirectly helping us for the completion of project and the resource provided.

Charuli Patil
182171006

ABSTRACT

Existing projects are based on granting and retrieving permissions to applications. When a user installs an application it grants permission to it and once granted it can use it without any limit, unless the user retrieves the permission manually. This is the reason for security to privacy issues because a harmless permit can sometimes be harmful in some cases. Security risks are highly dependent on the current situation and, while accessing a particular service is safe in some contexts, while it may violate privacy in another context. One of the solutions to this contextual control problem is the level of consent that allows the granting of strong and denied consent depending on the specific situation. However, manually adjusting the policies becomes more complex for the user as there are a number of policies that will need for the application to run properly. Therefore, we try to propose a context-based application that can provide flexibility to provide and recover permissions based on different contexts. We use the Android platform to demonstrate our work. In this model, Android applications are assigned contexts, where the contexts grant or revoke different permissions and permission-related conditions.

As mobile devices are increasingly used in various daily activities, they have become a movable storage that stores various personal/business information of users. Most mobile OSes, including Android, store personal data in databases and provide APIs for apps, which can be used to access a database managed by the system or to manage its own database. However, Android, which is our main focus here, stores data as plain text in its database; as a result, the database content can be leaked unintentionally through several vulnerabilities. Additionally, the responsibility for the security of the database content created by an app lies with the developer of the app, while the mobile OS only provides minimal security features such as isolation and access control.

Key Words: Android, access control, context based access control, user privacy, database security, location.

Contents

List of Figures	vi
List of Tables	1
1 Introduction	2
1.1 Background	3
1.2 User Privacy	4
1.3 Database Security	4
1.4 Organization of Report	4
2 Literature Survey	6
2.1 Android System Overview	6
2.2 Android Kernel	7
2.3 Libraries	7
2.4 Android Runtime	7
2.5 Application Framework	8
2.6 Applications	8
2.7 Android System Security	8
2.8 Mandatory Application Sandbox	8
2.9 Secure Inter-Process Communication	8
2.10 Application Signing	9
2.11 Android Permission System	9
2.11.1 Permissions	10
2.11.2 Permission Protection Levels	11
2.11.3 Permission Grouping	11
2.11.4 Permission Enforcement	12
2.12 Access Controls	12
2.13 Device Administrator	14
2.13.1 Device Admin Policies	14
2.14 Database Encryption Algorithms	14
2.15 Gap Analysis	15
2.16 Technologies Used	16
2.16.1 GeoFencing	16
2.16.2 Firebase	16
2.16.3 Sqlite Database	17
2.16.4 SqlCipher	17
3 Problem Statement and System Analysis	19
3.1 Motivation	19
3.2 Problem Statement	19
3.3 Objectives	19
3.4 System Management Plan	20
3.4.1 Project Overview	20

3.4.2	Project Deliverables	20
3.5	Project Management Plan	20
3.5.1	Description	20
3.5.2	Deliverables and Milestones	21
3.6	System Requirements	22
3.6.1	Hardware Requirements	22
3.6.2	Software Requirements	22
4	Proposed System	23
4.1	Proposed System Architecture	23
4.2	Modules	24
4.2.1	Modules Description	24
4.3	Operating System and API	25
4.4	Permission System	25
4.5	Android Application Components	25
4.6	Intents	25
4.7	Location Positioning Methods	26
4.7.1	GPS Trilateration	26
4.7.2	Wi-Fi Positioning Method	26
4.8	Design of Architecture	27
4.8.1	Camera Disable Module	27
4.8.2	Database Security Module	28
4.9	Control Flow Diagram	30
4.10	Defining Contexts and Associating Contexts	32
4.11	Methodology	34
4.11.1	Android Studio	34
4.11.2	Activity in Android	34
5	Implementation	35
5.1	Visualisation Tools	35
5.1.1	Android Studio	35
5.1.2	Installing Android Studio	35
5.2	Location Tracking	42
5.3	Camera Permissions	44
5.4	Database Security	45
5.5	Main Code	46
5.6	Implementation	48
5.6.1	Code Implementation	48
6	Results	53
7	Conclusion	61
7.1	Conclusion	61
7.2	Future Scope	61
	Appendices	62
A	Deployment Diagram	63
7.2	Publications	64
	Bibliography	66

List of Figures

2.1	High-Level Picture of Android Architecture Layers	6
2.2	Detailed Android Architecture Layers	7
2.3	Android User Permission Approvals	10
3.1	Milestones and Timelines	21
4.1	Access control framework	23
4.2	Location Capturing Phase	26
4.3	Location Detection Phase	27
4.4	Context-aware Android Role-Based Access Control (CA-ARBAC) components	27
4.5	Database Architecture	28
4.6	Control flow diagram	31
5.1	Download Android Studio	35
5.2	Setup Guide	36
5.3	Setup Completed	36
5.4	Start Installation	37
5.5	Import Settings	37
5.6	Android Studio Opening	38
5.7	Downloading SDK	38
5.8	Start Window	39
5.9	Install Window	39
5.10	UI Theme	40
5.11	Settings	40
5.12	Downloading Components	41
5.13	Downloading Components -II	41
5.14	Start New Project	42
5.15	Location Permissions	43
5.16	Getting Current Position	43
5.17	Main Activity	44
5.18	Defining Policies for Application	44
5.19	Defining Policies for Application	45
5.20	Defining Policies for Application	45
5.21	Defining Policies for Application	46
5.22	Code I	46
5.23	Code II	47
5.24	Code III	47
5.25	Getting Users Location	49
5.26	Device Admin Permission	50
5.27	Camera Disabled	51
5.28	SqliteDatabaseBrowser Software	52
6.1	Getting Location Permissions	53

6.2	Device Admin Activation	54
6.3	Camera not disabled	55
6.4	Camera Disabled	56
6.5	Registration Page for Admin	57
6.6	Registration Page for User	58
6.7	Code III	59
6.8	Code III	60

List of Tables

1.1	Context aspect and their states	3
4.1	Associated Contexts	33

Chapter 1

Introduction

The changing technology in mobile phone has introduced new smart technology that offers emerging yet sophisticated capabilities. In addition, smart phones are increasing the access to devices works with apps such as GPS, SMS, Wi-Fi etc. As a result, several challenges occur mainly in security issues. The most important feature is to limit the performance of users using apps and its services. In the meantime, the existing security and privacy control system hosts a built-in and imperfect security model. Most current applications work with device-level security that is the basic of each application, especially for installation. A user installing third-party applications thinks that the applications will not misuse the device resources. Similarly, if a user wants to use that program, he or she must grant all permission requests. This whole or empty decision leads to a built-in access control industry. In addition, as long as the user grants the permissions, there is no way to limit or revoke these permissions based on user activity. For example, a user may want to limit the amount of SMS sent on a day for saving a bill using status information such as access patterns. To address these issues, our model proposes an access control mechanism managed by the access control component i.e. Context-Related Role-Based Access Control (CtRBAC) [2]. Contexts are defined as a set of actions or obligations associated with a specific function. Policy restrictions are defined according to the resources, services, and permissions that are granted to apps.

The main issue with the latest operating system is that once permissions are granted to an application, it always has the right to access the services without taking prior permissions from the users [1]. Operating systems do not support a change in application permissions based on user context. This perspective of application permissions is very risky in case of mobile applications. The threat arises when an application acts as a virus and uses device resources to spy or leak the user's personal data without consent. Even when the users are carrying smartphones in public places, this may expose their data while being unaware of the harmful acts going on their devices [6].

The need for user-friendly data-based policies goes from high profile employees to ordinary smartphone users. For example, government employers, such as national laboratories, are prohibiting their employees from bringing any camera-enabled device to the workplace, including smartphones. But employees may need to have their own devices at all times as their devices may contain data and services they may need at any time. With content-based device policies, employees may be allowed to use smartphones to prevent apps from using the camera and any device resources and rights that they are allowed to block. In contrast, at work, while the user device can retain all its original privileges outside the work area. Content-based policies are also a political imperative with law enforcement personnel who need to disable the camera, microphone, and location services from their android devices during secret meetings while maintaining these services in non-confidential environments. With context-sensitive policies, users can specify when and where their applications can access their device data and resources, which reduces the chance of hijackers accessing such information.

Many mobile OSs, including Android, store personal information and provide application APIs, which can be used to access system-managed data or to manage its own database [7]. For Android, it stores

data precisely, but the data may be unintentionally charged with several risks. The security of content information created by the application lies with the developer of the app, while the portable OS only provides some security features such as fragmentation and access control. Google introduced Full Disk Encryption (FDE) for Android having versions 3.0 and above it. It provides confidential encryption and decryption that works for the file system for all device read / write functions. But, FDE protects data only when data it is encrypted. When a user unlocks an Android device, all information on the device is decrypted and announced. That means, FDE protects data only on a locked device from offline attacks and not while being active. Once a device is unlocked, anyone with proper permissions can access, use and modify all the data in the database even if FDE is used [8].

1.1 Background

Adjusted policy limits are defined in terms of device resources, services, and permissions granted to applications during installation. Such policies specify what services the device offers and restrict access to the device and user information. Policy restrictions are contextual and adjustable by the device user. We define a theme based on location and time. Location is determined mainly using Wi-Fi (A.P.) physical location, and their associated signal values allow us to distinguish between adjacent areas within the same work area, in addition to G.P.S. and connecting cell phones by phoning whenever available. We use the C.B.A.C. our policies on the Android app and include a tool that allows users to define virtual environments such as home or work using captured Wi-Fi parameters. Users can be more precise by distinguishing between areas within the same area, such as conference rooms. When a user prepares device policies that define the rights and resources of the service depending on the context, the policies can be applied automatically whenever the user is in a predefined location and timeout period.

The mobile app security focuses on preventing applications from accessing sensitive data and resources. Still, most have no effective means of enforcing those restrictions based on the standard-setting that separates them from adjacent subareas. Also, most of this work is focused on developing policy policies that do not restrict usage rights and apply only to the entire system. Thus, users disable all applications from using the camera and any device resources and rights that employers are denied while at work. At the same time, the tool can retain all of its original rights outside of the workplace. A CBAC requires the identification of context in a system, where the contexts can be defined as a set of actions or obligations associated with a specific task. Users are given the authority to change context as per requirement. Context aspect values are illustrated in table 1.

Context Aspects	1	2	3	4	5
Location	Undefined	Home	Work	Public	
Time	Morning	Noon	Afternoon	Evening	Night
Speed	Still	Low	Medium	High	
Network	Undefined	Home Wlan	Work Wlan	GSM Public	Hotspot
Noise	Silence	Low	Medium	High	
Light	Darkness	Low	Medium	High	

Table 1.1: Context aspect and their states

SQLite stores the entire database (definitions, tables, indices, and the data itself) as a single cross-platform file on a host machine. It implements this simple design by locking the entire database file during writing. SQLite read operations can be multitasked, though writes can only be performed sequentially.

Due to the server-less design, SQLite applications require less configuration than client-server databases. SQLite is called zero-conf[9] because it does not require service management (such as startup scripts) or access control based on GRANT and passwords. Access control is handled by means of file system permissions given to the database file itself. Databases in client-server systems use file system permissions which give access to the database files only to the daemon process.

Another implication of the serverless design is that several processes may not be able to write to the database file. In server-based databases, several writers will all connect to the same daemon, which is able to handle its locks internally. SQLite on the other hand has to rely on file-system locks. It has less knowledge of the other processes that are accessing the database at the same time. Therefore, SQLite is not the preferred choice for write-intensive deployments.[10] However, for simple queries with little concurrency, SQLite performance profits from avoiding the overhead of passing its data to another process.

1.2 User Privacy

User grant permissions to applications. But these permissions granted are harmless at a time but can be harmful in some cases. Permission to access camera in normal areas (Home, Public Area etc.) is harmless but in a military or sensitive area it can be dangerous. These permission therefore should not be granted in such areas.

Applications shouldn't be able to disclose the location or time of the device to third party sources. Applications should not allow third party apps to fake a location. Applications should not generate a fake time and gain access to certain content that is not allowed. [1]

1.3 Database Security

When we are trying to build an app, securing data is really important. The biggest issue is that it doesn't use Android key store which is a safe way to manage keys and basically prevent someone from getting access to the data that we've encrypted. Google has launched Jetpack security, which basically allows us to easily encrypt user's data, files, and shared preferences without having to really understand the ins and outs of security. If we're building an applications that runs on a rooted or compromised device, it means the user is present on this device and the file system is unlocked. Once the file system is unlocked and even though we have full disk encryption, the data is now available to an attacker or someone that's getting the device. Secondly, attacker might actually have data that we don't want others to see. So if we have API keys, tokens, things like that to authorize services, we probably don't want those to get leaked, and if we probably don't want anyone to get access to that because technically, they could start pretending to be user, and start using our quota and getting free things that we have to pay for [8].

1.4 Organization of Report

The organization of the report gives a brief introduction to the structure and chapters of this thesis.

Chapter 1, "Introduction" gives the overview of the different concepts and terminologies used in this thesis.

Chapter 2, "Literature Survey" gives an overview of the current research that has been done in the related fields of this thesis.

Chapter 3, "Problem Statement and System Analysis" includes the motivation behind the project along

with the actual project problem statement formulation and scope of research.

Chapter 4, "Architecture and Methodology" includes the research that is already done, different algorithms used in this. Chapter 5, "Proposed System" contains a more detailed sight about the system architecture, flow of system, method used and data set information.

Chapter 6, "Implementation" gives the details about implementation of different modules alongside software and hardware requirements.

Chapter 7, "Results and Analysis" present the results of the system proposed.

Chapter 8, "Conclusion and Future scope" draws conclusion and outline the future direction of work.

Chapter 2

Literature Survey

It is important to recognize the previous research done in regard to this field to be able to correctly advance in the right direction. Previous work on mobile app security (O.S.) focuses on preventing apps from accessing sensitive data and resources, but most have no effective means of enforcing those restrictions based on well-established conditions that differentiate between adjacent subareas. Also, most of this work is focused on developing policy policies that do not limit the rights peruse and apply only to the whole system. Also, existing policy programs do not cover all the possible ways in which applications can access user data and device resources. Finally, the localized policy policies are not good enough to distinguish between adjacent areas without Hardware or local devices. In many cases, those applications take the context as they are provided without providing or testing the context-sensitive methods of mobile devices.

In this paper, we discuss the most promising solution for user privacy as well as database security.

2.1 Android System Overview

Android is a complete software stack consisting of different layers. It is based on the Linux kernel. It is developed by the Open Handset Alliance (OHA), which is led by Google. Android system is made up of four layers; each layer is manifesting well-defined behaviour and providing specific services to the layer above it, as shown below in the figure. A more detailed figure showing the components inside each layer is also shown in the figure:

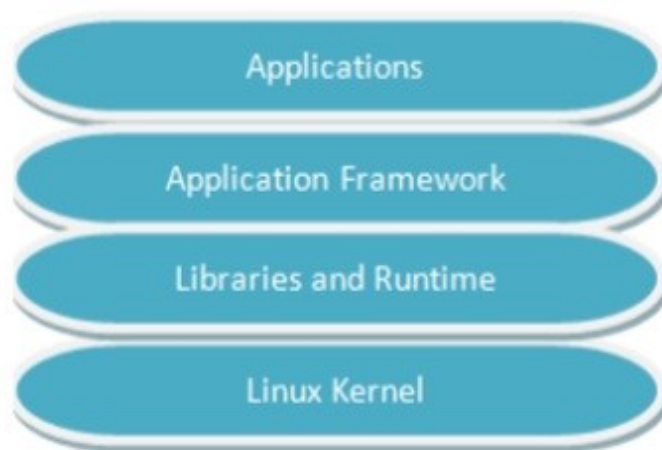


Figure 2.1: High-Level Picture of Android Architecture Layers

2.2 Android Kernel

The Android Kernel is the first layer of the Android system that interacts with the device hardware. This layer acts as a bridge that connects the device hardware and the Android software layers above the kernel. Android Kernel is a modification of the traditional Linux Kernel for an embedded environment. Android Kernel has also made many enhancements to the original Linux Kernel. Android kernel takes care of duties such as process management, memory management, device drivers, networking, power, and security.

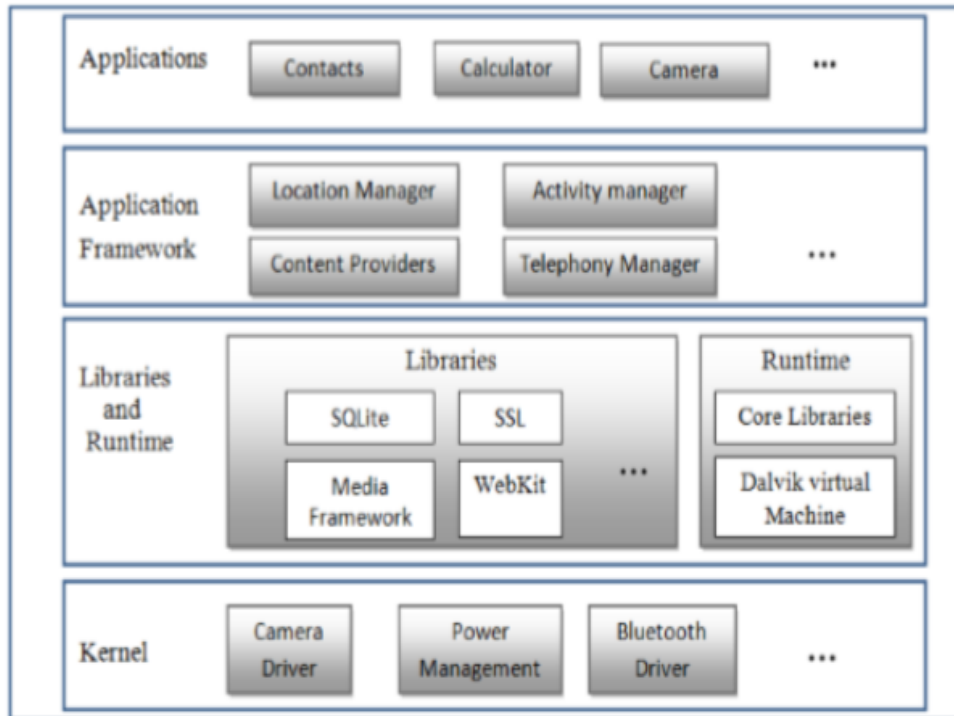


Figure 2.2: Detailed Android Architecture Layers

2.3 Libraries

The libraries component consists of a set of C and C++ libraries used by different components of the Android system. The layer, also called the native layer, causes libraries to be written in C/C++ and optimized for the hardware. They are opposed to the Android applications and frameworks, which are written in Java. It is a translation layer between the kernel and the application framework. Developers use these libraries through the Android application framework. Android applications can access native capabilities through Java Native Interface (JNI) calls.

2.4 Android Runtime

The Android Runtime has two parts: the Dalvik Virtual Machine (D.V.M.) and Java Core Libraries. The D.V.M. executes java class files compiled into index format. It is analogous to the Java Virtual Machine (JVM) that exists on personal computers and servers today. In android, every application runs in an isolated process inside a separate Dalvik virtual machine instance allocated for that application. The Dalvik V.M. relies on the Linux kernel for providing lower-level functionality (e.g., memory management). Android relies on Java core Libraries for most of the services related to Java programming language. Java core libraries depend on the service they get from the kernel and Dalvik V.M.

2.5 Application Framework

The application framework provides a collection of services or systems for the developer to be used when writing applications. The presence of the application framework simplifies the reuse of components. The services in the application framework publish interfaces for different functions so that any third-party applications can then use those functions without the need to reinvent the wheel.

2.6 Applications

The application layer of the Android operating system is the closest to the end-user. By default, android comes with a rich set of applications, including the browser, the SMS program, the calendar, the e-mail client, maps, Contact Manager, an audio player, and so forth. User applications also belong to this layer.

2.7 Android System Security

Android is a truly open mobile platform. To secure an open platform, we require a robust security architecture. Android is considered as one of the most secure and flexible operating systems for mobile platforms. Android security mechanism is a multi-layered security system that is designed by imitating traditional Linux system security approaches. Hence it provides similar security services such as protecting user data, system resources, and providing application isolation. It is designed with both developers and device users in mind. Android achieves these security objectives based on the following key security features provided by the Linux kernel:

1. Mandatory application sandbox (process isolation)
2. A secure inter-process communication mechanism
3. User-based Permission model
4. Cryptography
5. Mandatory access control using Security-Enhanced Linux in Android (SEAndroid)

2.8 Mandatory Application Sandbox

Android makes use of one of the critical security features (user-based security model) provided by Linux kernel is to provide a mandatory application sandbox. The user-based security model allows application resources to be identified and kept isolated. The Application Sandbox is performed in the Linux Kernel. Since the Application Sandbox is implemented in the lowest level of the Android software stack (in the kernel), this security model is also applicable to native code applications and to operating system applications. The application sand boxing encompasses all user applications, the application framework, the application runtime, and operating system libraries. With the exception of the kernel and some operating system code running as root, all of the other components run within the application sandbox. Moreover, each component above considers that the parts beneath it are trusted and properly secured. A given application or component is sandboxed by executing it in a separate process with a unique user I.D. (UID). Each application is assigned its own set of private data structures and is prevented from interfering with other processes execution or from performing sensitive operations such as accessing the recording audio, making phone calls, or receiving SMS messages. The private data structures are also labeled the applications UID.

2.9 Secure Inter-Process Communication

In addition to any Linux-type Interprocess communication methods such as local sockets, file systems, and signals, android provides different kinds of new I.P.C. mechanisms such as Binders, Services, Content Providers, and Intents. Application sandbox can talk to other applications via such kinds of secure I.P.C.

mechanisms. Binder: Binder is a type of remote procedure call (R.P.C.) mechanism in android. It allows communication between processes in the same application and also between processes in different applications. A traditional Linux driver is used to implement it.

Services: Services are one of the Android application components that run in the background to accomplish long-lasting tasks. Services also expose R.P.C. communication interfaces that are reachable through binders. **Intent:** Intent is a lightweight messaging object used by applications to tell other applications that they want to do something. In other words, it is a notification message from one process to another to accomplish some work. **Content Providers:** A Content Provider is a way of having access to data stored on the device. Applications get access to data exposed by other applications through Content Providers. An application can also define its own Content Providers to expose its data to other applications. Although it is possible to implement inter-process communication using Linux I.P.C. mechanisms such as network sockets and world-writable files, the new Android I.P.C. mechanisms are the recommended ones.

2.10 Application Signing

All Android applications running on the Android system must be signed by the developer before they can be installed. An android system refuses the installation of applications that are not signed. The primary purpose of application signing is to distinguish applications from one to another. Developers sign their applications with their own private keys (self-signed). Even though it is allowed, it is not a requirement to sign Android applications with a certificate authority, which also shows that no applications are trusted. Android system may not be able to place an application in an Application Sandbox unless the application is signed correctly.

The Android system uses the signed certificate to determine which application is represented by which user id. With application signing, since different applications run under different user I.D.s, one application cannot access private data of other applications except through one of the secure I.P.C. mechanisms discussed earlier. When an Android application is going to be installed, the new application can choose to share a UID with other applications already installed on the device. Applications ask to share UID with other applications by specifying it in their manifest file. In such cases, the Android system checks if the public key certificate of the requesting applications matches the key used to sign the other application installed on the device. If the two keys are the same, the two applications are then treated as being the same application regarding security issues. Besides this, Applications signed with the same key can also have different Application Sandboxes and UIDs with the option to share security permissions at the signature protection level. Therefore, the Android system also uses Application signing to allow and or deny access to signature protection level permissions.

2.11 Android Permission System

As mentioned earlier, all user applications installed on the Android system run in their own sandbox. Hence, by default, an application is only allowed to access data in its sandbox (its own private data) and a limited range of less sensitive resources such as the internet. The permissions associated with such kinds of less sensitive resources are automatically granted at install time, and the user won't be able to cancel them later. These permissions are called Normal Permissions. For all other resources and services, applications access to resources is performed through a strictly controlled access procedure which is managed by Android system security. The access controls are implemented in two ways. Some functions are totally not allowed to be used by user applications. For example, there is no way to access the SIM card directly. In other cases, the sensitive APIs are allowed for trusted applications and are protected through a security mechanism known as Permissions. Some out of the many protected APIs include: Telephony functions, SMS/MMS functions, Camera functions, Location data (G.P.S.), Bluetooth functions and Network/data connections. To be able to access such kinds of security and privacy sensitive resources other than their own private data directories, applications need to be granted different application layer permissions by the user. Applications have to request the permissions explicitly they need in order to use those permissions and execute successfully. They do so by declaring the required permissions in the manifest file as shown in figure.

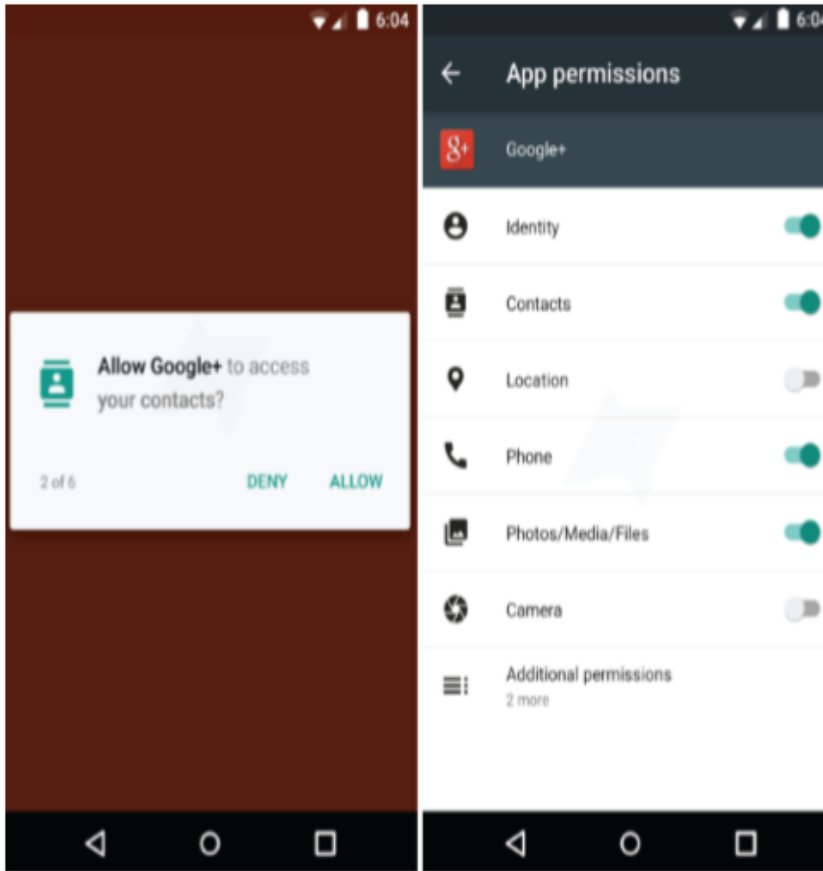


Figure 2.3: Android User Permission Approvals

User approval is required before an application can get access to critical operations (e.g. making calls, sending SMS messages). Starting from Android version 6, user approval is done during runtime. When an application is going to perform some operation that requires a given permission (more accurately group of permissions), android prompts the user to either allow or reject the requested permission/permissions. The left side of figure shows a screen shot of permission user approval in android. Furthermore, the user can later modify permissions (revoke previously allowed permissions or allow new permissions). The right side of the same figure shows run time permission modification. Once the user has granted permission, the permissions are applied to the application as long as the application is not uninstalled. Permissions granted to an application are removed if the application is uninstalled. Therefore, re-installation of the application results in requesting all of the permissions again. If the application attempts to use a sensitive resource or service without declaring the necessary permission in its manifest, a security exception is thrown back to the application.

2.11.1 Permissions

Android defines a set of core permissions for protecting O.S. Resources and services. These are called system-built-in permissions. Android defines around 140 system built-in permissions. All of the system-built-in permissions in android are listed in Android developer's website. Some examples of system-built permissions are CALL PHONE; INTERNET; CAMERA, READ CONTACTS; READ LOGS; READ SMS; RECEIVE SMS; SEND SMS, WRITE SMS and so. System-built permissions provide a means to get access to restricted content and APIs. Third-party application developers can also define new permissions that are enforced using the same mechanisms as system-built permissions. We call these later ones as user-defined

permissions or custom permissions. User-defined permissions are used by third-party application developers to protect their applications/components from other applications. However, some device capabilities are not available to developers such as the ability to send SMS broadcast intents because these permissions are signature level permissions. The main concern of this paper is about Android system built in permissions and not about user-defined permissions. Therefore, from now on, we simply use the word permission to refer to android system-built permissions.

2.11.2 Permission Protection Levels

Android classifies permissions into four levels depending on the potential risk that may come from granting a given permission. This classification helps the system in determining whether or not to grant the permission to an application requesting. The four protection levels are described below. **Standard Permissions:** Normal permissions are permissions that are considered to have relatively minimal risk to the system, other applications or the user. Permissions such as ACCESS_NETWORK_STATE; CHANGE_NETWORK_STATE; INTERNET and SET TIME_ZONE are categorized under standard permissions. Appendix A shows the list of all normal permissions in android. Applications require standard permissions to access data or resources outside their sandbox/private data. Permissions with average protection level are automatically granted to a requesting application at installation time without explicitly asking approval from the user. For example, if an application declares in its manifest file that it needs INTERNET, Android system automatically grants INTERNET permission to the application at installation time. **Dangerous Permissions:** These are risky permissions that could potentially harm the operation of other applications or user's data if granted. Examples of dangerous permissions are CONTACTS, LOCATION, PHONE and SMS. Android system does not automatically grant dangerous permissions to a requesting application because it could introduce a negative impact on the private user information or on the device. To get access to dangerous permissions, applications should declare in their manifest file that they need the permission and the user has to grant the permission explicitly. Dangerous permissions requested by an application are displayed to the user during run time for confirmation before allowing the application to use it. The exhaustive list of dangerous permissions is listed in Appendix B. **Signature:** As explained previously, Android system grants permissions declared by one application to another application if the two applications are signed with the same certificate. Permissions achieved this way are called signature permissions.

Android system automatically grants permission without asking user's explicit approval if the certificates of the two applications matched. **Signature or System:** Similar to the Signature protection level, applications that are signed with the same certificate as the application that are declared the permission are automatically granted the permission. The difference between Signature and Signature or System protection levels is that Signature or System can be requested by an app that comes with the Android system image. Hence, Signature or System protection level permissions can be granted to Android operating system applications even though they are not signed with the same certificate as the application declaring the permission. The "Signature Or System" permission is used for specific situations where multiple devices have apps built into a system image and have to share specific features explicitly because they are being built together. In addition to the above four divisions, there are also some individual permissions in android that are very sensitive and are not allowed for most applications. SYSTEMALERTWINDOW and WRITESETTINGS are two examples of individual permissions. If an application wants to use one of the individual permissions, it should first declare the permission in the manifest. In addition, it should create an intent that is used to request confirmation from the user. The Android system displays authorization screen to the user for approval.

2.11.3 Permission Grouping

Android system permissions are grouped into related permissions. For example, the following four permissions are grouped under the PHONE permission group:- -Directly call phone numbers; this may cost you money -Write call log (example: call history) -Read call log-Reroute outgoing calls -Modify phone state -Make calls without your intervention The list of Android permission groups is listed below. Starting from Android version 6.0 (API level 23), permission requests are presented to the user in the form of permission groups and not as a single permission. As previously stated, android system automatically grants Normal

permissions to a requesting application. However, a dangerous permission must be approved by the user before granted. When an application requests a dangerous permission that belongs to some permission group that is declared in its manifest file, Android system performs one of these two things: - If the application had previously been granted one or more other permissions in that permission group, the system automatically grants the permission without informing the user. For instance, let say an application which had previously requested and been allowed WRITECALENDAR permission requests READCALENDAR permission, then the system immediately grants the READCALENDAR permission. However, if the application does not have any prior granted permission in that permission group, the system displays a dialog box to the user for approval. The authentication window shows a description of the permission group that the requested permission belongs but does not show a description of the specific requested permission within that group. As an example, if an application requests the READCALENDAR permission, the system displays an authentication window saying the application needs access to your CALENDAR. The system grants the permission only if the user accepts it.

2.11.4 Permission Enforcement

As stated previously, in android, every time the user installs application, a unique user I.D. is generated for the application and the application is assigned that UID. The application runs under the UID. All data stored by that app is assigned the same UID, whether a file, database, or other resource. Every application sandbox accesses its own private resources by direct system call to kernel. The Linux Kernel permissions on private resources for that application are set to allow full permission by default. However, an app needs access from the user to use resources which are not private. An app can access resources without their private identifier using two different methods. First, when an application is assigned to low-cost public resources such as SD Card and CAMERA permissions, it is placed on a Linux group that has access to compatible resources. Therefore, the application was assigned to the group I.D. (GID) over UID. Such types of public resources are also available by directly interacting with the kernel underlying system calls in the same way as accessing. private resources. Linux Kernel enforces access control policy i.e. control access to the file system ensures that the application has the required permissions. E.g. checks whether the application is allowed to open file in CAMERA by checking the application GID and GID privileged CAMERA access. File system access control uses the traditional Linux Discretionary Access Control. Linux Kernel Login Control also supports a Mandatory Access Control (M.A.C.) program called Security Enhanced Android (SEAndroid).

2.12 Access Controls

The access that a user has to a particular resource starts with authorization. This is the process of ensuring that only authorized rights are exercised by a user. That's the policy enforcement. And then there needs to be some method of determining what those rights are. That's the policy definition. In most environments, there needs to be some type of rights that a user will obtain using an access control model. And different organizations have different access control models, depending on what their overall goals are for this access control. There are different access controls used for privacy.

1. **Role-based access control models:** MPDROID is good example of pure RBAC model that is close to our approach. It is a security framework that supports two kinds of access control models at two layers of Android system: RBAC at the application framework layer and mandatory access control (MAC) at the kernel layer. At the application framework layer, it enhances APS with RBAC to provide fine-grained access control. This enables users to define their own security policy and control malicious applications. At the kernel layer, it implements MAC to allow administrators enforce fine-grained access control. Administrators can limit activities of applications and their processes according to a centralized security policy. Similar to our system, users authorize Android applications by assigning roles instead of permissions. But MPDROID does not take context into account.
2. **Context-aware access control models.** Various kinds of CAAC models have been proposed in the past. Four of the most recent ones are ConUcon, CRêPE, and ConXsense. Our CAAC policy model is analogous. Similar to our proposed system, it associates Android permissions with context. However,

it works only for two kinds of contexts: location and time. Our system is designed to support more types of context. In addition, is a pure CAAC model unlike CA-ARBAC that is a hybrid of RBAC and CAAC. ConUcon proposed a general CAAC framework that works with several mobile platforms. It uses context information to protect privacy and to control resource usage. It supports active context usage control, that is, context check is not only performed prior to resource access but also during the access. CRêPE developed a system that enforces fine-grained context-related policies on Android. In CRêPE, policies can be configured both by phone users and authorized third parties locally or remotely, via SMS, MMS, Bluetooth, or QR-code. The policies can also be applied in a system-wide manner. ConXsense focuses on usability of APS that is ignored by most other CAAC models. It does not require users to configure policies. Instead, it is based on a probabilistic approach that automatically classifies contexts according to their security and privacy risks using machine learning and context sensing.

3. **Context-aware role-based access control** models. There exist also works that combine aspects of RBAC model with context awareness. Some of the existing literature that fall under this category are dynamic role-based access control for Android , context related role-based access control , CA-RBAC and role-based access control for Android (RBACA).
4. **Dynamic role-based access control** for Android offers an RBAC system that is similar to traditional desktop computers. It allows the management of multiple users on Android mobile devices. In addition, it provides both application and permission level fine-grained access control by using RBAC.
5. **Context-related role-based access control** incorporates the contextual information of user and system environment with the traditional RBAC. Context-related role based access control defines user roles based on their access privileges. Users are categorized according to their access rights of device's resources and services. Each user possesses one role at a time.
6. **Context-aware role-based access control** also proposes an access control model that combines RBAC with context awareness. CA-RBAC dynamically assigns roles to users and permissions to roles according to the current context. CA-RBAC is similar to our system in that it assigns permissions dynamically. However, CA-RBAC is for ubiquitous computing environments not for Android. In addition, our system does not change roles contextually to avoid unnecessary creation of roles.
7. **Mandatory Access Control (MAC)** This means that the operating system is going to provide the limits on how much access someone will have to a particular object. And these are usually based on certain clearance levels. Every object that someone may need to access needs to be assigned a label. It's either confidential, or secret, or top secret, or some other type of label. The users then are provided with certain rights. Certain users can access confidential objects. Other users may be able to access top secret objects. And it's the administrator that determines what specific access role a particular user has. The users can generally not change any of these access roles under a mandatory access control model.
8. **Discretionary access control (DAC)**, is an access control model that most people will probably be familiar with because it's used in most operating systems that we use today. For example, if you create a spreadsheet, you as the owner get to determine who else in your environment has access and what type of access they have to that spreadsheet. And you as the owner of that spreadsheet can modify that access at any time. This makes for a very flexible access control model because you, as the owner of that file, can determine exactly who has the type of access. And you can make these changes whenever you'd like. But it's also a very weak form of access control security because it relies on the owner of the file to be setting the proper security for that particular object.
9. **Attribute-based access control (ABAC)**, allows us to create very complex relationships between the applications we're using and the data that is used by those applications. You can use many different types of criteria to determine exactly what type of access a person might have. You'll often see ABAC referred to as the next generation of authorization because of all the complex context that's associated with this authorization model. For example, ABAC authorization may consider your IP address, the time of day, your geographical location, and many other criteria to determine exactly what type of access you have.

2.13 Device Administrator

Android includes support for enterprise apps by offering the Android Device Administration API. The Device Administration API provides device administration features at the system level. These APIs allow you to create security-aware apps that are useful in enterprise settings, in which IT professionals require rich control over employee devices. For example, the built-in Android Email app has leveraged these APIs to improve Exchange support. Through the Email app, Exchange administrators can enforce password policies — including alphanumeric passwords or numeric PINs — across devices. Administrators can also remotely wipe (that is, restore factory defaults on) lost or stolen handsets. Exchange users can sync their email and calendar data. The Device Administration API to write device admin apps that users install on their devices. The device admin app enforces the desired policies.

2.13.1 Device Admin Policies

The Device Administration API supports the following policies:

1. **Password enabled:** Requires that devices ask for PIN or passwords.
2. **Minimum password length:** Set the required number of characters for the password. For example, you can require PIN or passwords to have at least six characters.
3. **Alphanumeric password required:** Requires that passwords have a combination of letters and numbers. They may include symbolic characters.
4. **Complex password required:** Requires that passwords must contain at least a letter, a numerical digit, and a special symbol. Introduced in Android 3.0.
5. **Require storage encryption:** Specifies that the storage area should be encrypted, if the device supports it. Introduced in Android 3.0.
6. **Disable camera:** Specifies that the camera should be disabled. Note that this doesn't have to be a permanent disabling. The camera can be enabled/disabled dynamically based on context, time, and so on. Introduced in Android 4.0.

2.14 Database Encryption Algorithms

It's possible to encrypt data at a number of levels, from the application to the database engine. For an MSP considering how to help a customer choose an encryption method, it's important to be clear on the purposes and requirements of these different encryption methods:

1. **API Method:** This is application-level encryption that is appropriate across any database product (Oracle, MSSQL, etc). Queries within the encrypted columns are modified within the application, requiring hands-on work. If a business has an abundance of data, this can be a time-consuming approach. Additionally, encryption that functions at the application level can lead to increased performance issues.
2. **Plug-In Method:** In this case, you'll attach an encryption module, or "package," onto the database management system. This method works independently of the application, requires less code management and modification, and is more flexible—you can apply this to both commercial and open-source databases. With this option, you will typically use column-level encryption.
3. **TDE Method:** Transparent data encryption (TDE) executes encryption and decryption within the database engine itself. This method doesn't require code modification of the database or application and is easier for admins to manage. Since it's a particularly popular method of database encryption, TDE is explored in further detail below.

4. **Android Keystore System:** Benefit of using android Keystore is you don't have to hardcode the key for encryption and decryption. Post encrypting the raw data we can save it either in SharedPreferences or the Database. We can only encrypt the content of the SharedPreferences and same with the SQLite database. Since androidx crypto library introduced we can encrypt the shared preference file also using that library.
5. **SQLCipher** is another library which helps us adding one more layer of security by encrypting the entire database itself and we save encrypted content using Android Keystore. SQLCipher is an SQLite extension that provides transparent 256-bit AES encryption of database files. Earlier SQLCipher which is Open Source Full Database Encryption for SQLite was not available for android. But now it's available for android platform

2.15 Gap Analysis

From the survey done, it is found that previous work on mobile app security (O.S.) focuses on preventing apps from accessing sensitive data and resources, but most have no effective means of enforcing those restrictions based on well-established conditions that differentiate between adjacent subareas. Also, most of this work is focused on developing policy policies that do not limit the rights peruse and apply only to the whole system. Also, existing policy programs do not cover all the possible ways in which applications can access user data and device resources. Finally, the localized policy policies are not good enough to distinguish between adjacent areas without Hardware or local devices. In many cases, those applications take the context as they are provided without providing or testing the context-sensitive methods of mobile devices.

Saint proposed to enhance the security mechanism of the Android by improving install and runtime policies. Communication between applications or components is subjected to security policies asserted by both the caller and callee applications. However, device security is controlled by the application provider's policy and not by the user's policy. Therefore it's only suitable for a developer. Moreover, it can pose only install-time policies.

Another work concerned about fine-grained access control is Scandroid, which is a tool for Android applications' automated security certification. It statically analyzes data flows through Android applications and makes security-relevant decisions automatically. It is a reasonable model for offline certification. It only tries to improve the existing Android security mechanism to be best practice. It mainly depends on source code inspection and Android manifest file; thus, it is not applicable for the average user.

Apex proposed the Android permission extension framework to a comprehensive policy enforcement mechanism for the Android platform. Although the framework enables users to grant permissions in a fine-grained manner and enforces policies at runtime, explicit context is not taken into account. Thus generic context framework is still needed for more exceptional access control.

Also, the increased number of GNU GPL license applications results in a higher chance of installing Trojans and similar malware. W. Enck ET. Al. Propose Kirin security service for android, which performs lightweight certification of applications to mitigate malware at install time.

In Paranoid Android, an alternative solution is proposed where security checks are applied on remote security servers that host exact replicas of phones in virtual environments. It is a security model that performs attack detection on a remote server in the Cloud where the execution of software on the phone is mirrored in a virtual machine. Although it can deal with the security of applications, it mainly depends on the Cloud.

Finally, some researches have been carried out to model the context-aware system to provide meaningful and valuable context information rather than raw context data to the system. ContextDroid is presented, and it is designed to provide application developers with the services required to build a context-aware application to reduce overhead quickly. In Diffuser, a single user scenario of a mobile phone is presented.

It proposed differentiated user access control model which classifies smartphone user based on specific sets of user access privileges. However, it can't fix existing issues in access control of the smartphone system. Moreover, it can only define static runtime constraints since it doesn't consider a dynamic user context. ConUcon is a continuous usage control that extends the U.C.O.N. model and proposes a user interface to express policy. It exploits contextual information, such as time and spatial information. However, the policies are considered mostly from an application point of view and not from a user perspective.

2.16 Technologies Used

2.16.1 GeoFencing

Geofences are virtual fences or perimeters around the physical location geo fences detect when someone comes in or leaves the given region when this happens it is possible to trigger events and actions such as sending a notification to the users mobile phone. As a user moves around with her phone a new position is calculated every now and then from a variety of different sources such as Wi-Fi and cellular data GPS.

The phone uses the updated position to check if the user has entered or left any of the geofences and in this case it triggers the corresponding actions these actions can let the user know that they have entered the geofence for instance by displaying a notification. However they can also be silent without informing the user such would be the case if you wanted to track how many times the user has come into your shop and how much time they spend there. Geofences require no additional hardware or sensors to be installed. Bluetooth can help improve the accuracy of your geofences especially indoors for practically all cases though you will require that users have your mobile application or one of your partner's mobile applications installed on their device.

Geofences work both when the app is actively being used as when it's running in background mode for instance if your user has the phone inside their pocket however behavior will be slightly different for instance your app might receive less frequent location updates in order to preserve battery power behavior will also change according to the platform which is being used Android or iOS and the position technology used.

2.16.2 Firebase

Developing a successful app isn't easy. To reach a broad audience, you'll need to consider your IOS, Android, and mobile Web users. To build these platforms, we'll need a back-end server to store data and support the apps. As we would, want to get our users logged in, which means our back end will have to scale. And then after we've solved your scaling problems, we have to find more ways to spread the word to get new users.

Firebase has all the tools you need to build a successful app. It helps you reach new users, keep them engaged, scale up to meet that demand, in addition to getting paid. Firebase, has Test Lab and crash reporting to prevent and diagnose errors in our app. our back-end infrastructure problems are solved with our real-time database, file storage, and hosting solutions. Acquiring new users is easy with Invites, AdWords, and dynamic links. And using the authentication component, we can get those users logged in with minimal friction. Once installed, we can keep our users engaged with notifications, cloud messaging, and app indexing. Then, with Remote Config, we have the freedom to experiment with new features and optimize the user experience in real time. All-new Analytics component, designed uniquely for Firebase, brings insight into how well these components are working for we and our users. With Firebase Analytics, we can measure and optimize our advertising campaigns, discover who are our most valuable users, and understand exactly how they are using

our app. Now, all these components work great on their own and provide a solid infrastructure to build out our app, but they work even better when combined in creative ways.

2.16.3 Sqlite Database

It's an SQL database engine. SQLite is sort of it's very small, so it's very compact. Some of the features is designed to be embedded in applications or in hardware. It's serverless. It writes directly to the disk drive, as opposed to sending IPC to a server that takes care of storing things. A very small footprint. Fully configured it's less than 250 kilobytes in size. Generally small databases. It has a lot of interesting features.

It allows you to define your own SQL functions that are just C functions, or your own collating sequences, so if you're working in languages with an unusual collating sequence, you can accommodate that. The entire database sits in a single source file. The entire database sits in a single disk file, which is a cross platform. So you can write the database on a big endian machine, and then just copy it over to a little endian machine and it works fine. So it's been picked up and it's being used quite a bit. It supports a big subset of SQL. It gives you atomic or acid transactions, which is a very important feature. It's out there as public domain, so lots of people are using it that I don't know about.

SQLite is good for that people who have done SQL for a long time don't normally think of. Because these other four reasons down here, a client server enterprise database doesn't do these things very well, and so people don't normally think about those kinds of things. We can obviously use it for anything that you can use a client server database engine for, just substitute SQLite for the client server database. SQLite can be used as a stand-in for our big client server database during testing or demonstrations or development where you don't want to have the hassle of setting up the client server database. SQLite allows us to think in a much higher level in the sense that you don't have to worry so much about how you're accessing the data. You just need to tell SQL what data you want to access. And this, especially with scripting languages, this actually plays very well. You can embed SQLite in your application, and instead of building some hash table or using some B-tree to access your data. You just store it all on a database, and then you do a query to pull the data out that you need. Depending on what your application is, this can work really well.

2.16.4 SqlCipher

SQLCipher is a specialized build of the excellent SQLite database that performs transparent and on-the-fly encryption. Using SQLCipher, an application uses the standard SQLite API to manipulate tables using SQL. Behind the scenes the library silently manages the security aspects, making sure that data pages are encrypted and decrypted as they are written to and read from storage. SQLCipher is a SQLite extension that provides transparent 256-bit AES encryption of database files. To date, it has been open-sourced, sponsored and maintained by Zetetic LLC. In the mobile space, SQLCipher has enjoyed widespread use in Apple's iOS, as well as Nokia / QT for quite some time. Given that Android by default provides integrated support for SQLite databases, our goal was to create an almost identical API for SQLCipher, so that developers of all skill level could use it, without a steep learning curve.

SQLCipher is often the go to choice for Android apps for the following reasons:

- Strong encryption (256-bit AES)
- Mature technology
- Maintained and supported by its developers and the open source community
- Supports virtually the same API as standard Android database functions

Unfortunately there is no such thing as a free lunch, it is important to understand not only the benefits but also the consequences of switching to an encrypted database. Here are a few to consider:

- APK size will be increased substantially

- Database performance will decrease
- Secure management of the encryption key (or password) is required
- Use of strong encryption has legal considerations in some jurisdictions

Chapter 3

Problem Statement and System Analysis

3.1 Motivation

In an open-source environment, android encounters not only security but also privacy issues. Third-party developers can submit their applications to the Android Market, where users can download and install. While this provides excellent access to a variety of applications, it does present some significant problems. With the advancement of information technology people want to access data anywhere anytime. Mobile Communication has gradually taken over the world, changing trends from simple voice communication and text messages to complex data communication and mobile internet business. But in the technology oriented world, crime is also increasing at a very fast pace, so there is a need of security to be analysed frequently. Phone has become a necessity but as it is said with benefits comes flaws. Being a portable device, crime can be easily committed through user intentionally or unintentionally

3.2 Problem Statement

The project aims to propose an access control mechanism for Android applications that allow smartphone users to secure and set policies for their use of device resources and services in various locations. We also propose a security architecture to construct a secure database environment on Android. The proposed system also provides tight access control over a database by using a runtime information of an app. Through the C.B.A.C. machine, users can, for example, set limited app privileges when using a device at work, and device programs can gain their first rights when the phone is used at home. This change to the device rights is automatically applied as long as the user's service matches the user-defined policy context. The user can also specify a set of default policies to be used when the user is located in a predefined location. The model can be built in a secure way to create a secure data center on android. The proposed system also provides robust access over the data domain through the operating time of the application.

3.3 Objectives

Policy-based formulation of smartphones is challenging because they must fulfill the following requirements:

- Applications should not be able to specify the device's location or time, as they should not exceed the policy limits applied to the device in a particular case.
- Since users are considered mobile workers, policy restrictions should be applied automatically to the device as a change of device location.
- Location accuracy needs to be higher than historical accuracy by G.P.S., as we need to apply different policies in different locations or nearby areas developed within G.P.S. location.
- Enforcement of context-based policies should not require application developers to modify the source code or impose any additional requirements on their applications.

- The principle used should not cause significant delays in the service's operation, which may adversely affect the program's performance..
- To provide security to database so that phones don't get hacked.

3.4 System Management Plan

System management refers to enterprise-wide administration of distributed systems including (and commonly in practice) computer systems. Systems management is strongly influenced by network management initiatives in telecommunications. The application performance management technologies are now a subset of system management.

3.4.1 Project Overview

This project involves finding of efficient methods to optimize resources required and algorithms that will provide better accuracy. Database which will improve the system performance and provides superior results. The location tracking app that should give most accurate location. So, the system should take optimized time in producing results.

3.4.2 Project Deliverables

The project addressed solving of two issues, privacy of data so as the data does not get leaked. Database security so as to secure credentials for misuse of data.

3.5 Project Management Plan

Project Management is the practice of initiating, planning, executing, controlling and closing the work of a team to achieve specific goals and meet specific success criteria at the specified time. The primary challenge of the project management is to achieve all of the project goals within the given constraints. It may include a summary or a detailed document or baselines, subsidiary management plans, and other planning documents. This document defines the approach to be followed throughout the project development.

3.5.1 Description

Operating systems do not support a change in application permissions based on user context. This perspective of application permissions is very risky in case of mobile applications. The threat arises when an application acts as a virus and uses device resources to spy or leak the user's personal data without consent. Even when the users are carrying smartphones in public places, this may expose their data while being unaware of the harmful acts going on their devices [6]. Many mobile OSs, including Android, store personal information and provide application APIs, which can be used to access system-managed data or to manage its own database [7]. For Android, it stores data precisely, but the data may be unintentionally charged with several risks. The security of content information created by the application lies with the developer of the app, while the portable OS only provides some security features such as fragmentation and access control. Google introduced Full Disk Encryption (FDE) for Android having versions 3.0 and above it. It provides confidential encryption and decryption that works for the file system for all device read / write functions. But, FDE protects data only when data it is encrypted. When a user unlocks an Android device, all information on the device is decrypted and announced. That means, FDE protects data only on a locked device from offline attacks and not while being active. Once a device is unlocked, anyone with proper permissions can access, use and modify all the data in the database even if FDE is used [8].

3.5.2 Deliverables and Milestones

The project involves solving of two issues, privacy of data so as the data does not get leaked. Database security so as to secure credentials for misuse of data. If the location is not tracked properly than the whole app will collapse and the camera wont get disabled and this will directly affect the performance of the system. Major task involved in the project are:

- System Design
- Implementation
- Comparative Testing
- Visualization

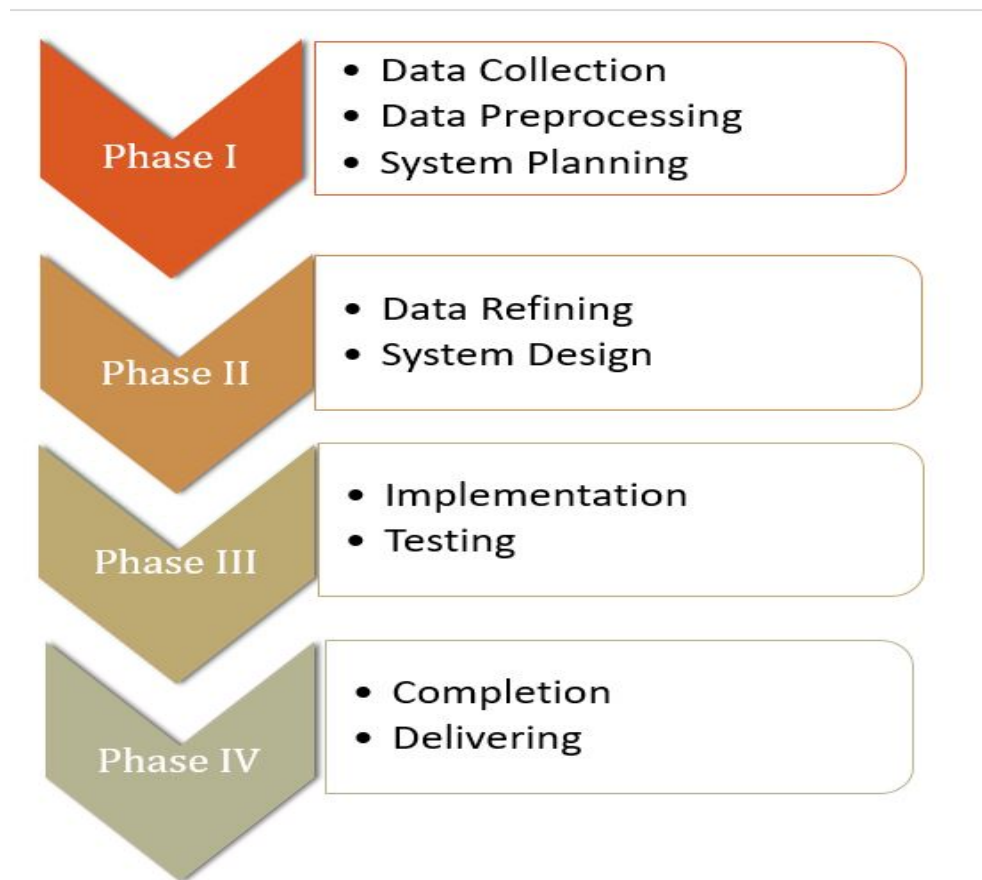


Figure 3.1: Milestones and Timelines

3.6 System Requirements

3.6.1 Hardware Requirements

- RAM : 13GB
- Disk Space : 108GB
- Operating System : Windows 10
- Memory : 12GB
- Processor : Intel(R) Xeon(R) with two cores @ 2.20GHz and 13 GB RAM

3.6.2 Software Requirements

- Android Studio : This is the main platform for developing application.
- Java : The whole app is created in android studio code using Java.
- Sqlite : This is used for creating and storing database.
- Sqlcipher : This is used to encrypt the stored data.

Chapter 4

Proposed System

4.1 Proposed System Architecture

A system architecture is the conceptual model that defines the structure,behaviour, and more view of the system. An architecture description is a formal description and representation of a system.

Going back to Android versions before Android version 4.3, Android permission system (A.P.S.) was not only coarse-grained but also static. By coarse grained, it means that the user has to accept all the permissions requested by the application during installation to be able to install the application. It was not possible to accept only some of the permissions and yet be able to install the application. Moreover, there was no way of revoking permissions later on during run time which makes it static. This weakness of the permission system had been the cause for a lot of over privileged applications that harm the privacy of the user. In response to these security and privacy risks, many researchers have tried to enhance Androids security system in different ways. Hence, the first generation of researches focused on making Android permission system fine-grained and dynamic. We studied some architectures and one of them we are going to implement more efficiently. Studied framework consists of an access control mechanism that deals with access, collection, storage, processing, and usage of context information and device policies.

To handle all the aforementioned functions, given framework design consists of four main components:

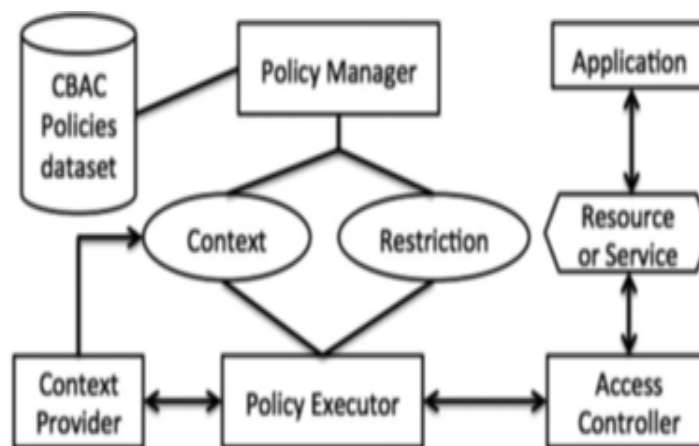


Figure 4.1: Access control framework

4.2 Modules

- Context Provider
- Access Controller
- Policy Manager
- Policy Executor

4.2.1 Modules Description

- **Context Provider:**

The Content Provider (CP) collects the physical location parameters (G.P.S., Cell I.D.s, Wi-Fi parameter) through the device sensors and maintains its database, linking each physical location with the user-defined location. It also validates and updates those fields whenever a call is retrieved.

- **Access Controller**

Access Control (A.C.) controls the authorization of applications and prevents unauthorized use of the device's services or resources. Or the Android O.S. has its own permission management system that considers that the app has rights to ask for help or resources, A.C. it complements the program with multiple control modes and specific use control permissions that better demonstrate application capabilities and limit its access to resources. The A.C. improves the security of the device system as the existing Android system has certain permissions, given the apps, it can give the apps more access than they need, potentially harmful code.

- **Policy Manager**

Policy Manager (PM) represent the interface used to create policies, accurately assigning application parameters to the content context. It specifically enables the user to configure which services and services are accessible through the application in the context provided by the C.P. For example, a user with PM can create a policy to allow local resources only when the user is active during the days between 8am 5pm.

- **Policy Executor**

Policy Manager (PE) enforces device restrictions by comparing device context with default policies. When an app asks access to a device, P examines user-defined restrictions that are set to PM to provide anything to deny system access, by submitting authorization information serves as policy enforcement to A.C. handle request requests and is responsible for resolving policy disputes and enforcing strict limits.

- **Access control principle:**

Suppose that parents want to control their children's phone usage. They do not want their children to spend too much time playing online games or accessing Internet. Thus they need means to control their children's smart phone access and determine what application can run and how long they can run.

- **Principle Privacy:**

A user might want to restrict the usage of phone resources and services according to the user role such as friend-using or admin.

- **Security Principle**

Consider as company's phones are handed out to employees for business use on which critical applications are installed to facilitate corporate operations and work owns. In the mean-time, company can prevent from disclosure of company's secret data via phones by limiting the number and capacity of SMS messages sent by employees each day. The second generation of researches started introducing context aware access control to the mobile environment. The majority of them are designed for all platforms in general and some of them are for android in particular.

4.3 Operating System and API

The Android operating systems are available on Linux based computers that enhance security and privacy support. Android has a security infrastructure with many components. This project provides developers with secure builds to design their applications. Android apps are bundled and run inside the Dalvik Virtual Machine, which separates individual operating systems and data. Each request is assigned to the user I.D. (UID) is the rights assigned to it, and is assigned to a dedicated component of its data file system. Request processes with different UIDs work differently and are separated from each other from the system. While apps have limited access to device services, developers can explicitly request access to their apps through the Android permission system.

4.4 Permission System

The Android permission system controls the privilege of accessing specific device resources and data of an app. Application developers that want to access protected android APIs need to specify the permissions they need in the AndroidManifest.xml file which, if in accurately assigned, can increase the risks of exposing the user. Each application lists the permissions listed in its Android-Manifest.xml file at the time of installation, and users have to grant all the requested permissions to proceed with the installation, or cancel the installation. The Android permission system don't allow users to grant or deny only some of the requested permissions, which limits the user's control of applications accessibility.

4.5 Android Application Components

Each and every Android app is made up of four main components: Functions, Services, Content Providers, and Broadcast Recipients. The function describes the user experience of the applications. The service component is used for background processing. The Content Provider acts as a global model for a specific application, so that all the applications in the system can use it. Stores and displays the shared data set of the owner application using the relational data interface. Ultimately the Broadcast Receiver component acts as mailboxes that allow applications to sign up for event or program events. All registered event recipients be notified by android in the event of this event.

4.6 Intents

Intents are asynchronous messages. They allow the three core application components Services, Activities, and Broad- cast Receivers to interact with each other and request functionality with each other. Interaction between elements occurs within your own processes or in external apps. Intents are used for different things such as data transfer, resource requests, work requests, service requests, and application action requests.

4.7 Location Positioning Methods

In our paper, we rely on Wi-Fi based placement strategies to determine device location. In addition to these strategies, we also collect location data obtained from Global Positioning System (G.P.S.) and cellular networks for situations where there is no Wi-Fi lending in areas of interest. In addition, we use the G.P.S. public location information in defining policy terms for areas that have not been visited before.

4.7.1 GPS Trilateration

G.P.S. is a positioning tool found on most smartphones that use data signatures from satellites to calculate device position. Data obtained from satellites contains the sending time stamp, the orbital information, and the position of the satellites. G.P.S. uses the trilateration method to calculate the devices location using at least three different satellite signals by measuring the satellite signals time difference or their received signal strengths. The location information provided from the G.P.S. includes the latitude, longitude. The accuracy is estimated to be in the range of 50 meters to 100 meters.

4.7.2 Wi-Fi Positioning Method

Many smartphone devices these days rely on Wi-Fi-based placement techniques because of its efficiency in locating a computer for a geographical area especially in areas where G.P.S. and mobile signals are weak or unavailable. These strategies are based on comparing devices that have access to Wi-Fi access points and fingerprints that have Wi-Fi access to known locations of the world. In our work, smartphone users describe their data for Wi-Fi A.P.s that include only user areas of interest. In addition to Wi-Fi A.P.s, we maintain the Received Signal Strength Indication (RSSI) for each A.P. to improve device accuracy, which, in our case, required differentiating between adjacent sub-locations.

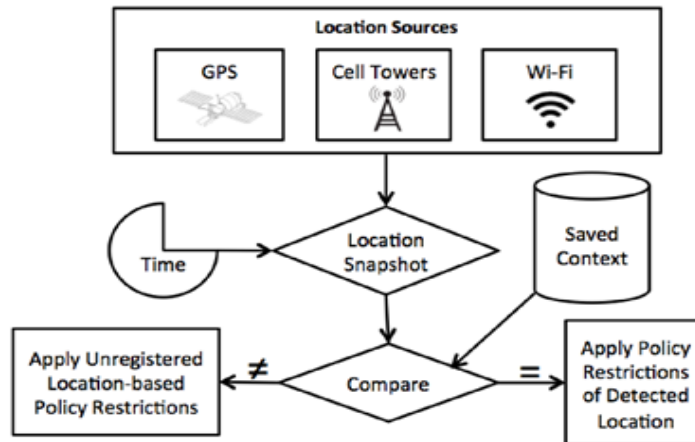


Figure 4.2: Location Capturing Phase

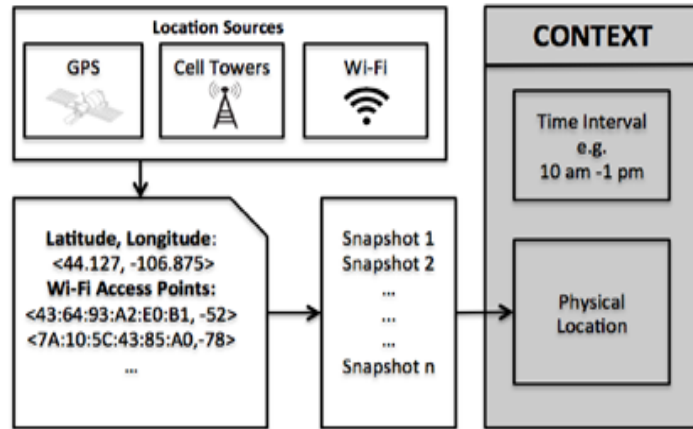


Figure 4.3: Location Detection Phase

4.8 Design of Architecture

4.8.1 Camera Disable Module

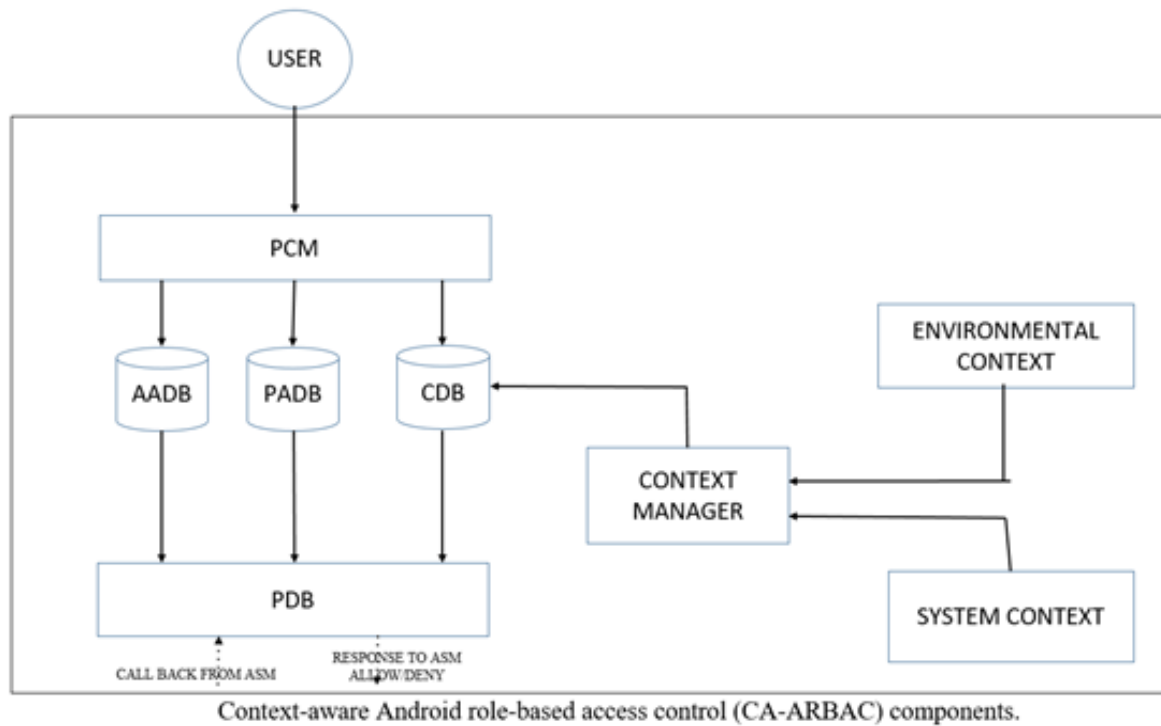


Figure 4.4: Context-aware Android Role-Based Access Control (CA-ARBAC) components

CA-ARBAC is responsible for applying only the policy. It is concerned with policy decision making, policy formulation, context acquisition, and policy retention. The policy emphasis is handled by the A.S.M. CA-ARBAC receives a call from A.S.M. where critical resources are available. CA-ARBAC has four components. The elements of the CA-ARBAC., shown in Figure 4.4, are described in the following sections as follows:

1. **Policy Decision Manager (PDM):** PDM is a key component of CA-ARBAC. It is this component that makes security decisions within CA-ARBAC. C.A.A.R.B.A.C. is connected to A.S.M. with PDM. PDM retrieves current domain information from the context database (C.D.B.). The C.D.B. maintains different types of status and its current values.
2. **Policy Enforcement Officer (P.C.M.):** Policies are prepared by the user through a user-friendly component called P.C.M.
3. **Content Manager:** The PDM needs access to the current status information to make an decision, which means, it needs to check whether the defined permissions-related context is fulfilled or not. Content Manager is part responsible for continuously receiving content updates for various content provider components and updating at new prices.
4. **Context-aware Android role-based access control policy databases:** Access control policies are stored in two different databases. - Application Allocation Data (AADB) stores the applications and roles associated with them. - Permission allocation data (PADB) is used to maintain assigned roles and permissions. P.A.D.B. and contains status information for related permissions.

4.8.2 Database Security Module

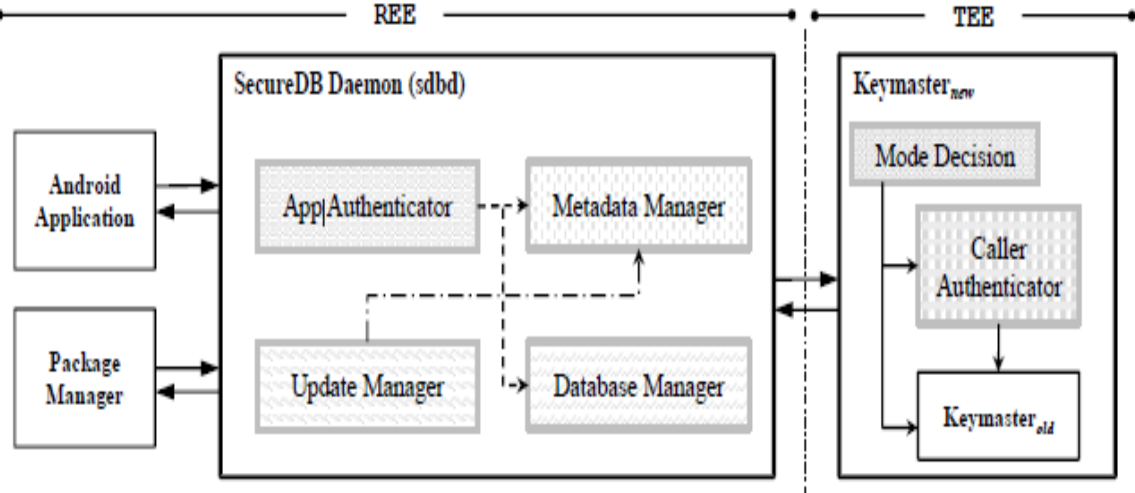


Figure 4.5: Database Architecture

- Keystore

KeyStore generates and stores a cryptographic key for sdbd in the TEE area. To ensure that this key is only used upon a request from sdbd, KeyStore verifies the identity of the caller process (i.e., sdbd). KeyStore consists of three modules: Mode Decision, Caller Authenticator, and Keymasterold as shown in the right-hand box of Figure 4.5. Keymasterold denotes the Keymaster of the current Android.

1. Mode Decision

For branching the way of Keymaster operation, we add a flag parameter that is utilized when KeyStore calls Keymaster(new). Depending on this flag, the Mode Decision of Keymaster(new) proceeds to Caller Authenticator or proceeds to Keymaster(old). That is, if KeyStore perceives the caller as sdbd, KeyStore sets the flag to authenticate the caller process, and Keymaster(new) proceeds to the Caller Authenticator module according to the Mode Decision. Otherwise, Keymaster(new) directly proceeds to Keymaster(old); in that case, it operates in the same manner as the current Android Keymaster.

2. Caller Authenticator

This module authenticates that the caller process is sdbd. To that end, we utilize a hash value of runtime information, which is related to a portion of sdbd, as a fingerprint. In more detail, the hash value is computed using a code segment that contains sections holding machine instructions as well as sections holding certain read-only data. Accordingly, unless sdbd is not modified, the code segment of sdbd remains unchanged. The fingerprint (hash value) is stored within an encrypted file that is managed by KeyStore.

• SecureDB Daemon

The SecureDB daemon (denoted as sdbd) is the main component in our architecture. It is started by init when the device boots. sdbd is a system daemon and it communicates with Android apps and PackageManager using UDS. sdbd consist of four modules: App Authenticator, Meta-data Manager, Update Manager, and Database Manager.

1. App Authenticator Module

This module authenticates an app connected to sdbd. When a connection is initially established between an app and sdbd, sdbd verifies the identity of the app. Please recall that the code segment of sdbd is used when KeyStore authenticates sdbd. However, sdbd can not authenticate the connected app in the same manner.

2. Metadata Manager Module

This module manages a meta-data file, which is related to the target application and its database. A metadata file stores all data used for app authentication and database operations. This file is named in the following form in a specific path that is managed by sdbd: [APP-UID][APP-PACKAGE-NAME].key. A meta-data file is generated by sdbd when an app requests opening a nonexistent database for the first time

3. Database Manager Module

This module is in charge of database operations and communications with an app. When an app requests a database operation and if a corresponding meta-data file exists, the App Authenticator Module compares the fingerprint of the requesting app process with the stored fingerprint. If the authentication is successful, sdbd and the app share a random session key (SK) using the Diffie-Hellman method

4. Update Manager Module

Unlike our assumption that an update of a well-structured sdbd is not required (see Section VIII-A), updates of apps occur frequently. If an app is updated, the fingerprint of the app, which is used for authentication, will be changed. Thus, sdbd must renew the stored fingerprint in a meta-data file while an app is updated. sdbd opens two types of sockets: one for apps, and one for PackageManager. The socket for apps is used to deal with database requests of an app. The socket for PackageManager is used to inform the update status for each other. While PackageManager updates an app, PackageManager connects to sdbd and informs which app is being updated.

4.9 Control Flow Diagram

The control flow diagram represents the actual flow of the project. Fig 4.5 shows the steps of the control flow diagram.

First the user need to activate device administrator.

If user wants to activate than following steps are followed:

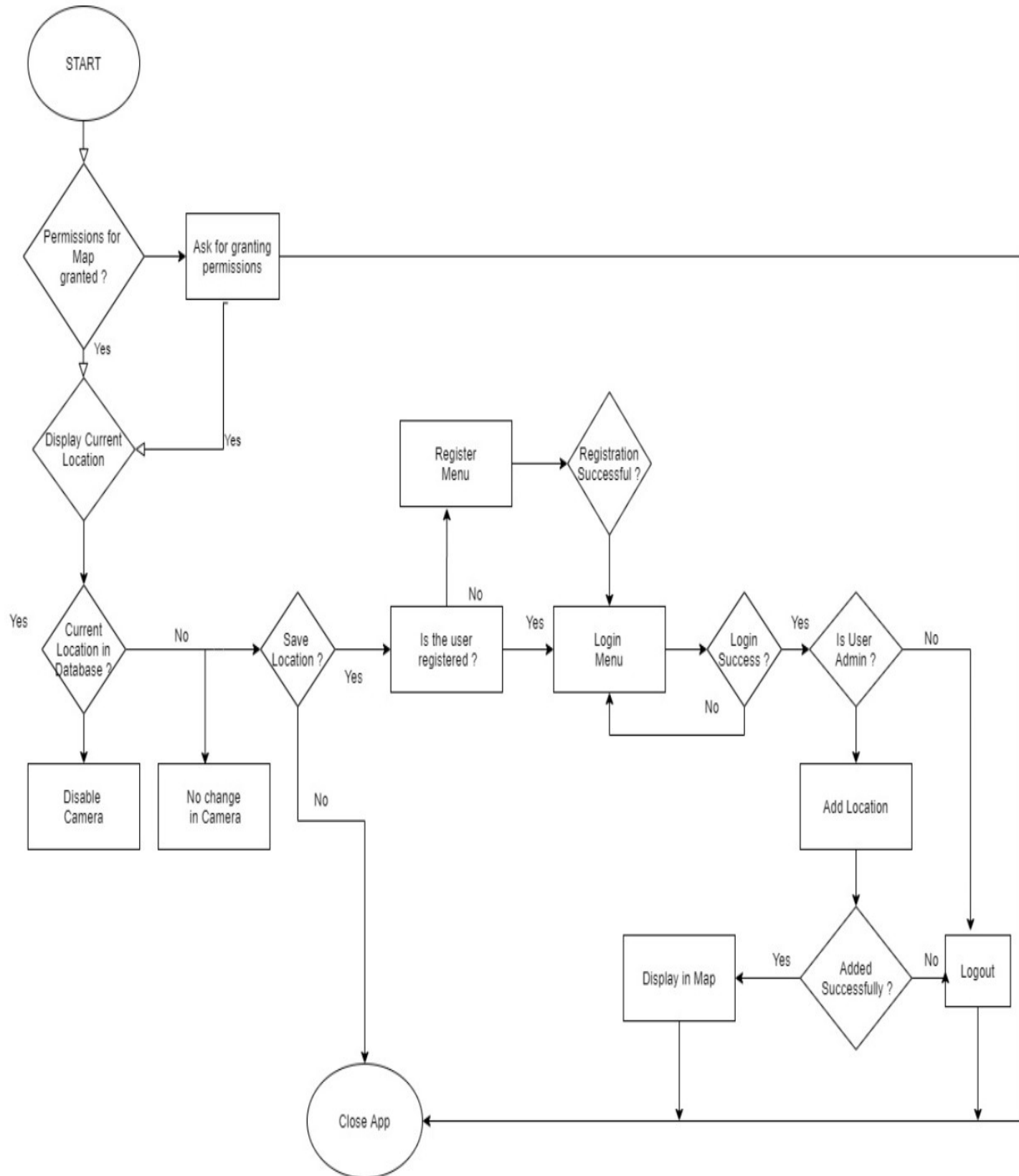


Figure 4.6: Control flow diagram

4.10 Defining Contexts and Associating Contexts

We have defined contexts with three of the permissions assigned to our roles as shown below

For example, the RECORD-AUDIO permission is the desired permission for multimedia and messenger applications. However, it can be hazardous in some cases such as when a user is in a meeting, or when a user is talking on the phone and when the phone is locked. Therefore, we set a policy that RECORD-AUDIO permissions are not allowed when the user is in one of these situations. To know if a user is in a meeting, we may need to know the location of the meeting and the time. Therefore, it is expressed using a combination of LOCATION and TIME conditions. For example, suppose the user has to attend meeting every Tuesday and Thursday in VJTI Computer Department from 2:30 pm to 4:30 pm and he does not want apps to record audio during the meeting.

The user can choose by specifying points on the map or by assigning the current location as a specified location. Preparation of the time context involves the choice of distance and dates. The previous context policy is represented in our program as `h ([LOCATION = 19.02, 72.85][TIME = 1430; 1630; TUESDAY, THURSDAY]), DENYING`. In addition, the user also does not want audio recording systems when talking on the phone or his phone is locked. The content of these two scenarios looks like this in our system `h [CALL-STATE = CALL-STATE-OFF-HOOK], DENYING` and `h [SCREEN-STATE = SCREEN-STATE-OFF], DENYING`, respectively. Similarly, the user can specify a context rule that denies permissions of CALL-PHONE, SEND-SMS, and RECEIVE-SMS permissions if phone is locked. Finally, the user stated that ACCESS-FINE-LOCATION and CAMERA permits were only allowed when he was away from his home such as in restaurants, markets, and work.

Context	Permissions	Context Policy	Context Policy Type
RECORDER	RECORD-AUDIO	USER IS IN A MEETING (LOCATION+TIME) USER IS TALKING ON THE PHONE PHONE IS LOCKED	DENY
MESSENGER	READ-CONTACTS	-	DENY
	WRITE-CONTACTS	-	DENY
	CALL-PHONE	PHONE IS LOCKED	DENY
	SEND-SMS	PHONE IS LOCKED	DENY
	RECEIVE-SMS	PHONE IS LOCKED	DENY
	READ-SMS	PHONE IS LOCKED	DENY
TRAVEL	INTERNET	-	-
	ACCESS_COARSE_LOCATION	USER IS NOT AT HOME	ALLOW
	ACCESS_FINE_LOCATION	USER IS NOT AT HOME	ALLOW
PHOTOGRAPHY	CAMERA	USER IS NOT AT HOME	ALLOW
	WRITE_EXTERNAL_STORAGE	-	-
	READ_EXTERNAL_STORAGE	-	-

Table 4.1: Associated Contexts

4.11 Methodology

The methodology include the detail information of the methods used in the project. The detail overview of:

4.11.1 Android Studio

- Android Studio is the official integrated development platform (IDE) of Google's Android app, is built on JetBrains' IntelliJ IDEA software and designed for Android development.
- It can be downloaded on Windows, Linux and macOS based system. It is the property of Eclipse Android Development Tools (ADT) as the primary IDE for traditional Android app development.

4.11.2 Activity in Android

- Activity is a java component in android with some predefined functions that are triggered in various application states, which we can pass through to do whatever we want.
- Activity class provides us with empty tasks that allow us to be in charge of everything

Chapter 5

Implementation

5.1 Visualisation Tools

5.1.1 Android Studio

Android Studio is the official integrated development environment (IDE) for Google’s Android operating system, built on JetBrains’ IntelliJ IDEA software and designed specifically for Android development. It is available for download on Windows, macOS and Linux based operating systems or as a subscription-based service in 2020. It is a replacement for the Eclipse Android Development Tools (ADT) as the primary IDE for native Android application development.

5.1.2 Installing Android Studio

- **Step – 1** : Click on the download android studio button

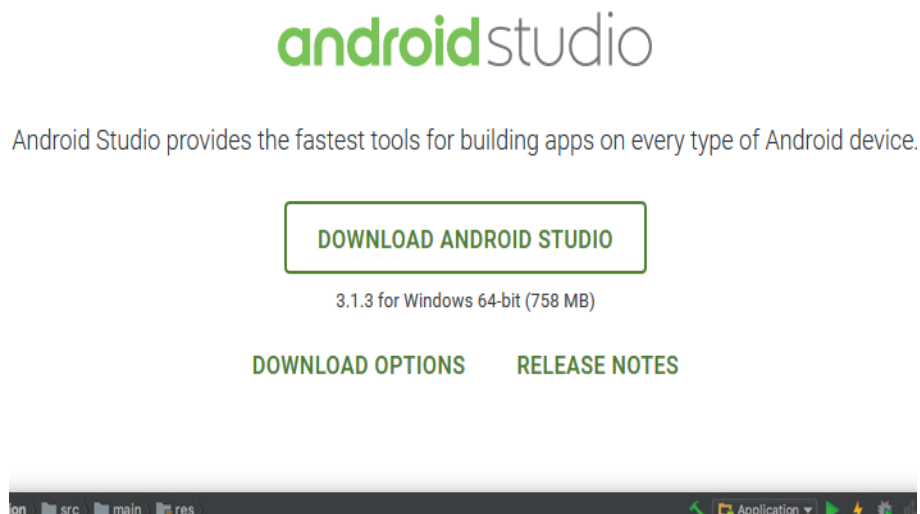


Figure 5.1: Download Android Studio

Click on the “I have read and agree with the above terms and conditions” checkbox followed by the download button. Click on Save file button in the appeared prompt box and the file will start downloading .

- **Step – 2** :After the downloading has finished, open the file from downloads and run it . It will prompt the following dialogue box . Click on next .

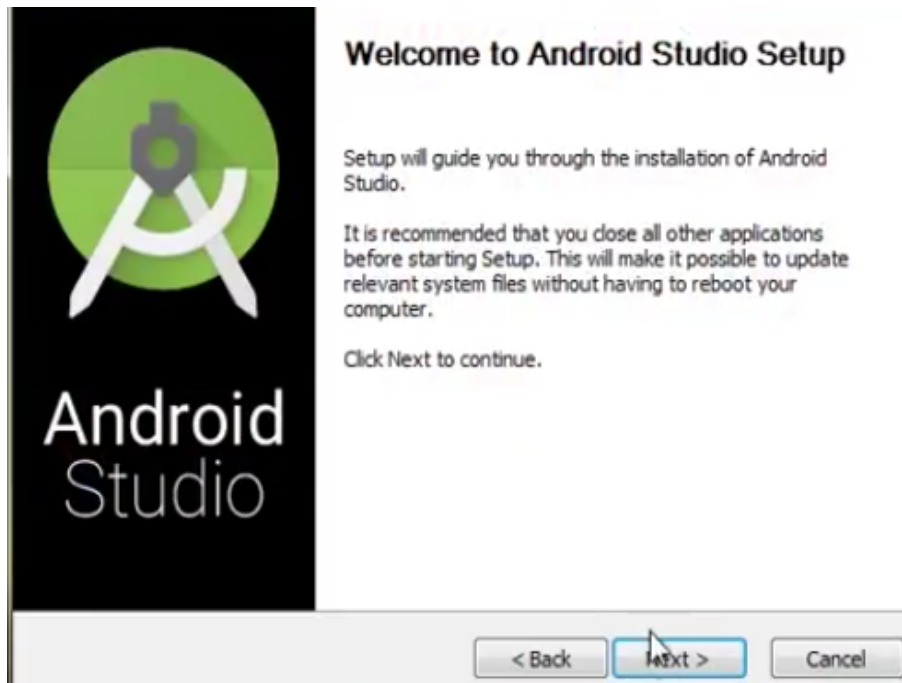


Figure 5.2: Setup Guide

In the next prompt it'll ask for a path for installation. Choose a path and hit next.

- **Step 3**:It will start the installation, and once it is completed, it will be like the image shown below.

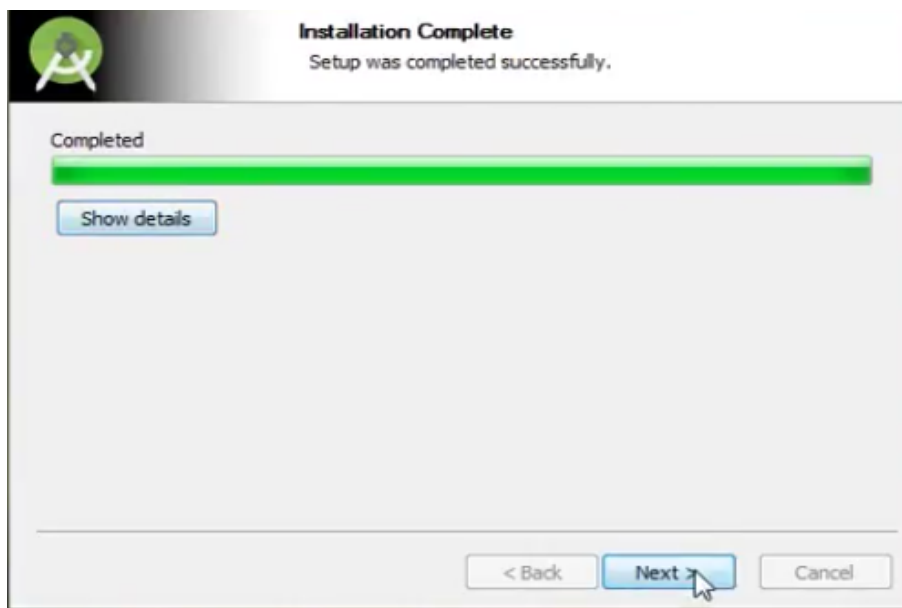


Figure 5.3: Setup Completed

Click on next



Figure 5.4: Start Installation

- **Step 4:** Once “Finish” is clicked, it will ask whether the previous settings needs to be imported [if android studio had been installed earlier], or not. It is better to choose the ‘Don’t import Settings option’.

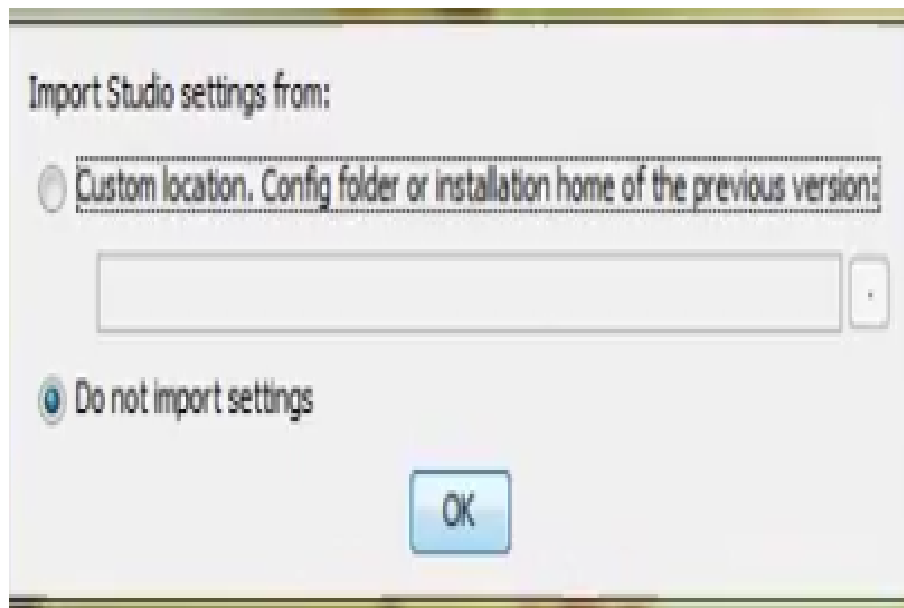


Figure 5.5: Import Settings

Click the OK button.

- **Step 5:** This will start the Android Studio.



Figure 5.6: Android Studio Opening

Meanwhile it will be finding the available SDK components .

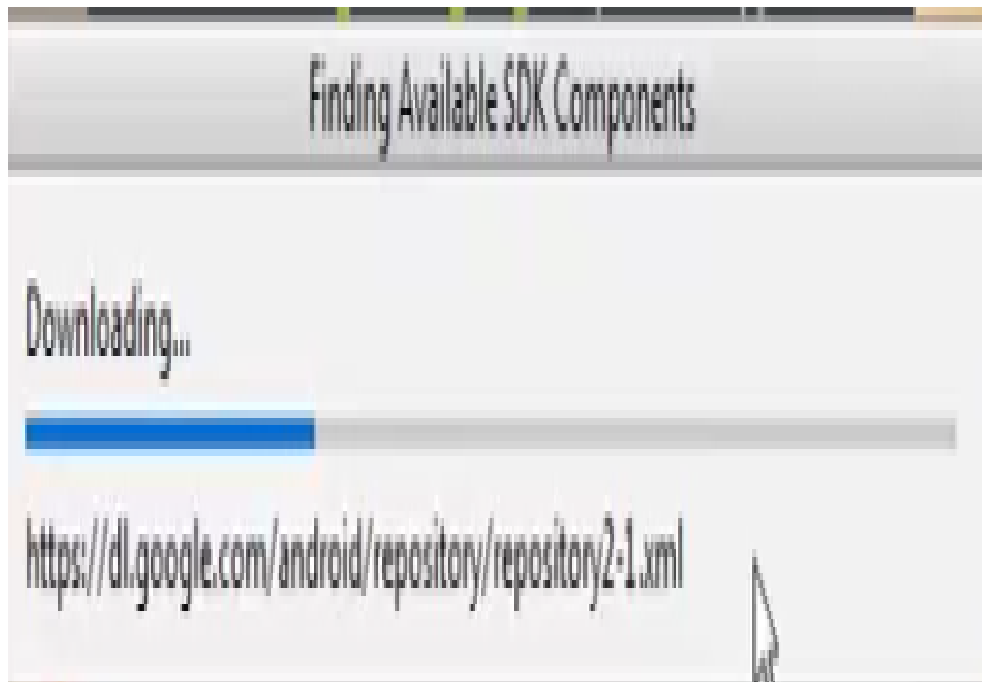


Figure 5.7: Downloading SDK

- **Step :6** After it has found the SDK components, it will redirect to the Welcome dialog box .
Click on next .

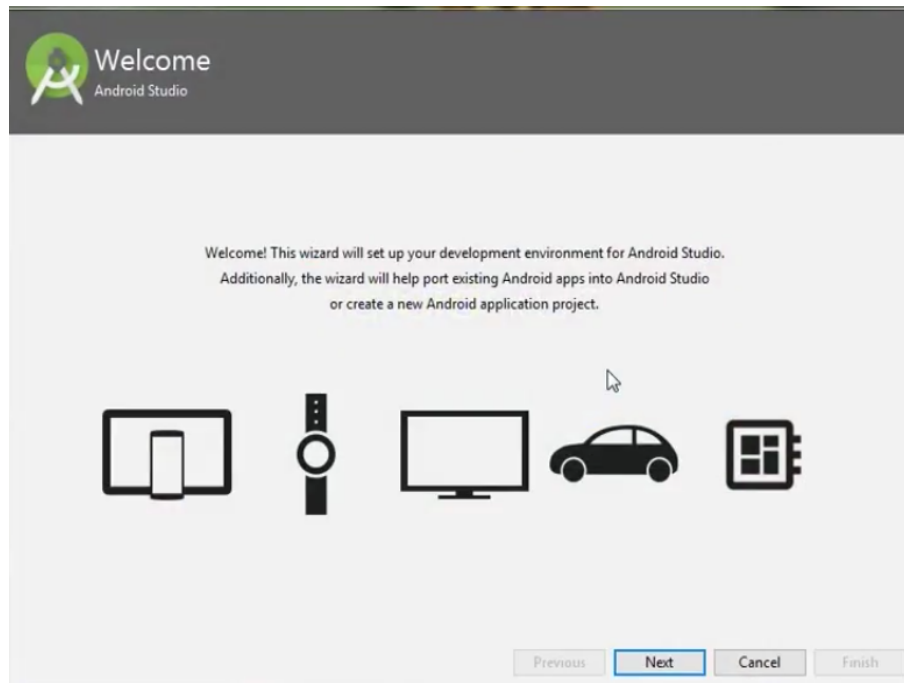


Figure 5.8: Start Window

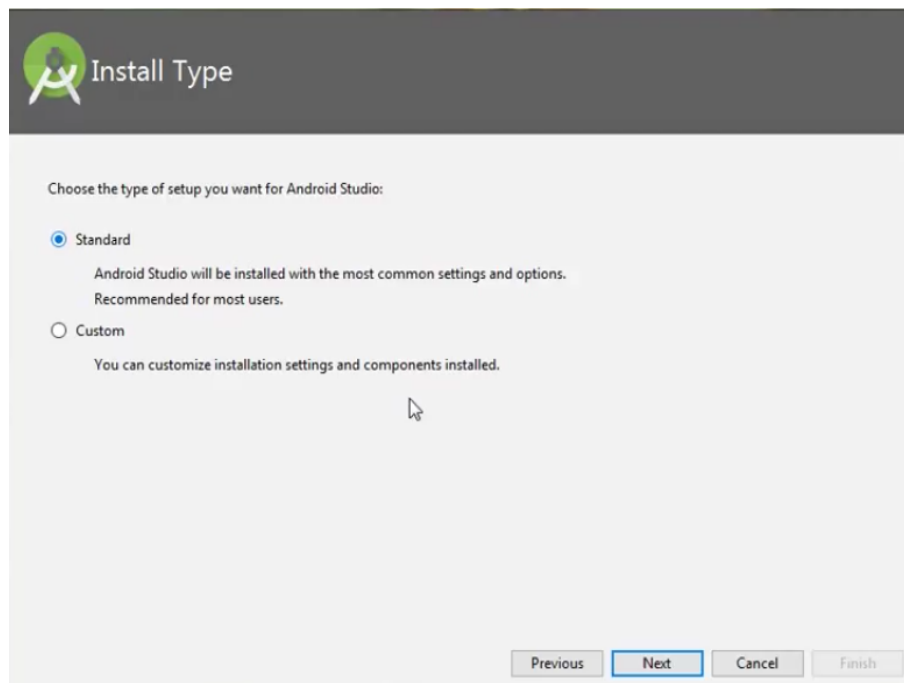


Figure 5.9: Install Window

Choose Standard and click on Next. Now choose the theme, whether Light theme or the Dark one . The light one is called the IntelliJ theme whereas the dark theme is called Darcula . Choose as required.

Click on the Next button

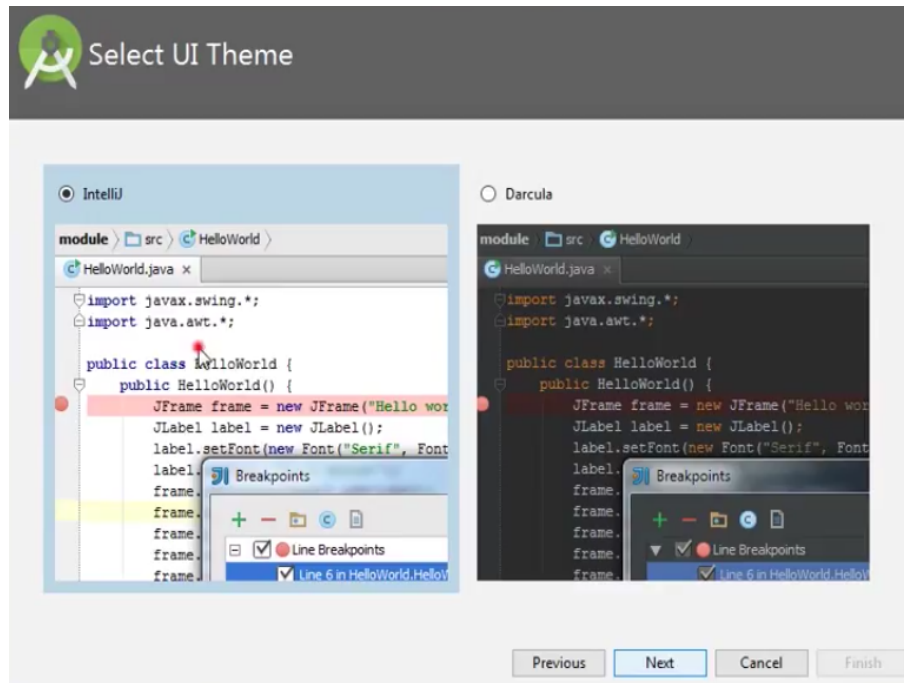


Figure 5.10: UI Theme

- **Step 8 :** Now it is time to download the SDK components

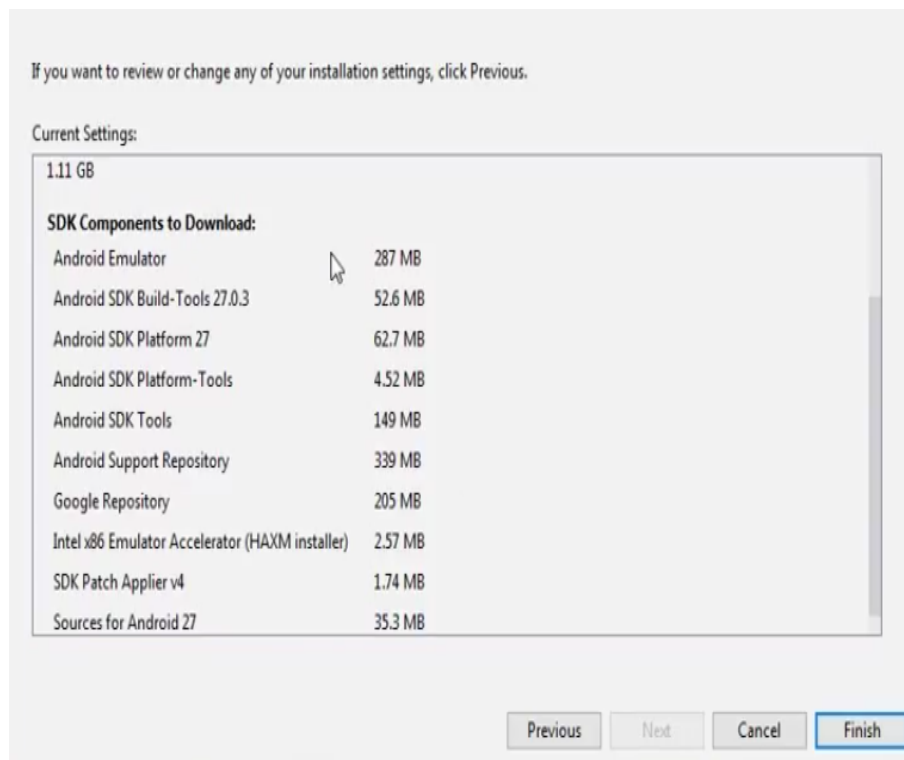


Figure 5.11: Settings

Click on Finish . It has started downloading the components

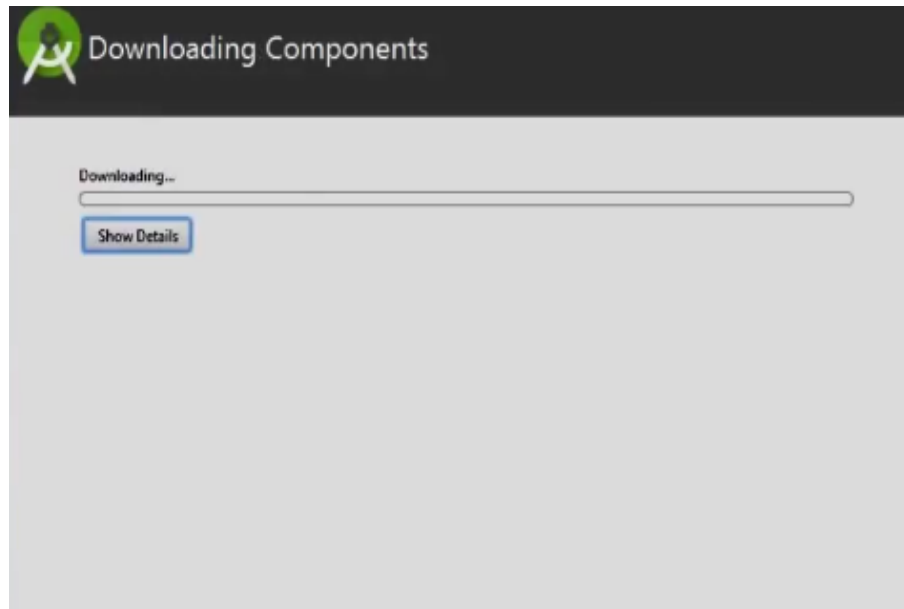


Figure 5.12: Downloading Components

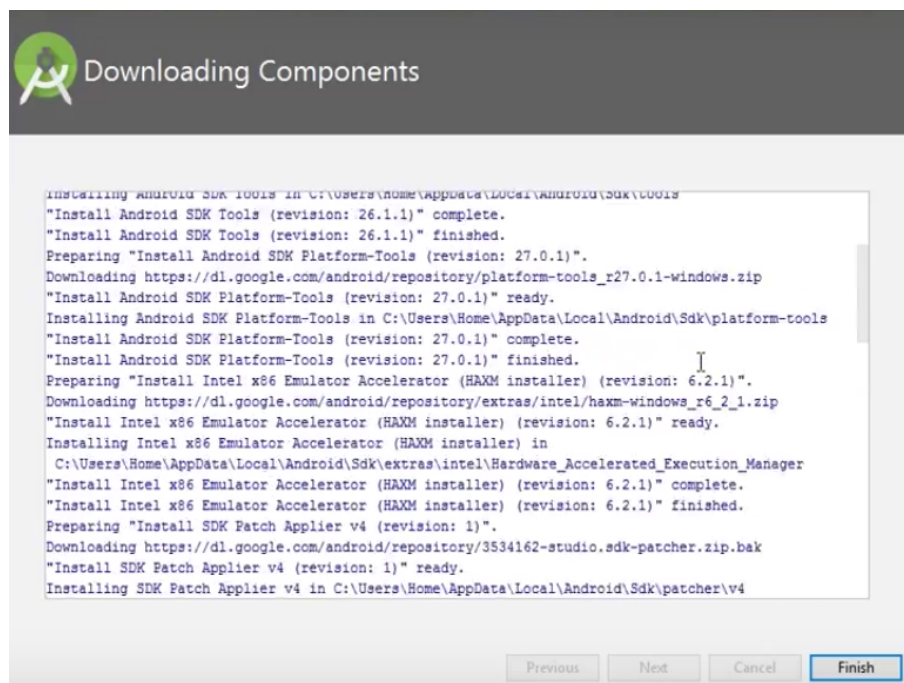


Figure 5.13: Downloading Components - II

The Android Studio has been successfully configured. Now it's time to launch and build apps. Click on the Finish button to launch it.

- **Step 9:** Click on 'Start new android project' to build a new app.

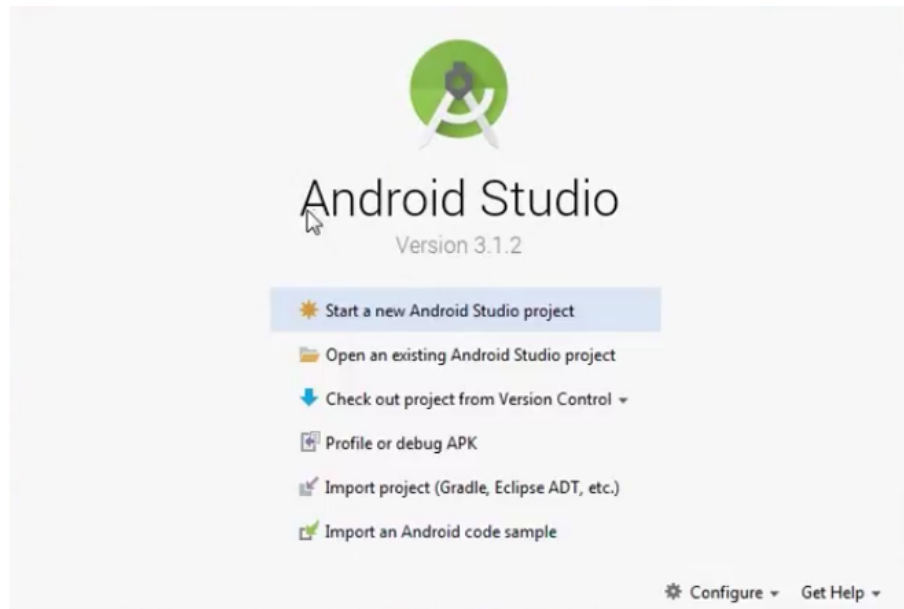


Figure 5.14: Start New Project

5.2 Location Tracking

- Map Activity in Android

- We need to add additional information to your AndroidManifest.xml file to use Google Maps.
- The user's current location can be found by using Google Map's API.
- This key is generated once and can be used for the further procedure
- Users take their devices anywhere and are regularly using them on the go, and as developers, we can capitalize on that by providing a more contextual experience based on their current location.
- We need to add additional information to your AndroidManifest.xml file to use Google Maps.
- The user's current location can be found by using Google Map's API.
- This key is generated once and can be used for the further procedure
- Users take their devices anywhere and are regularly using them on the go, and as developers, we can capitalize on that by providing a more contextual experience based on their current location.

- List of Permissions

- We need to add the following permissions into the AndroidManifest.xml
`<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />`
- It allows the API to determine a precise location from the available location providers, including the Global Positioning System (G.P.S.) as well as Wi-Fi and mobile cell data.
`<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />`
- It allows the API to use Wi-Fi or mobile cell data (or both) to find the device's location. The API gives us the location with an accuracy approximately equivalent to a city block.

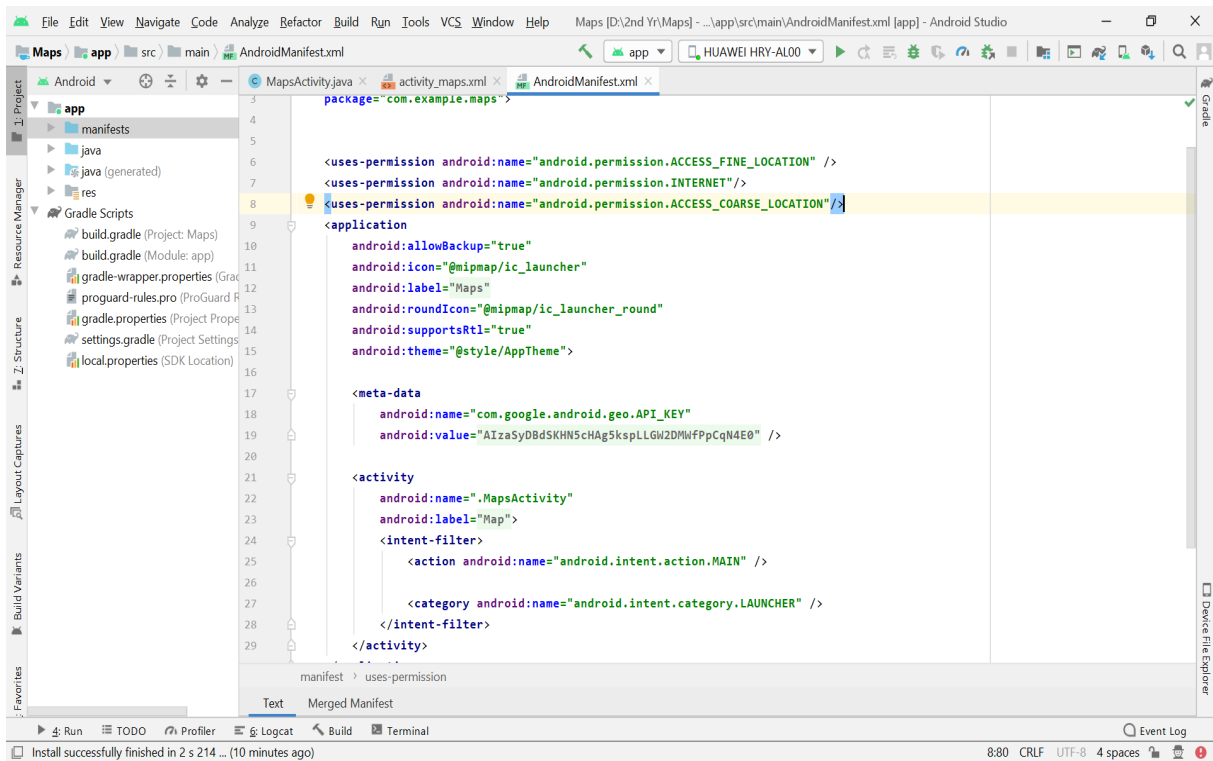


Figure 5.15: Location Permissions

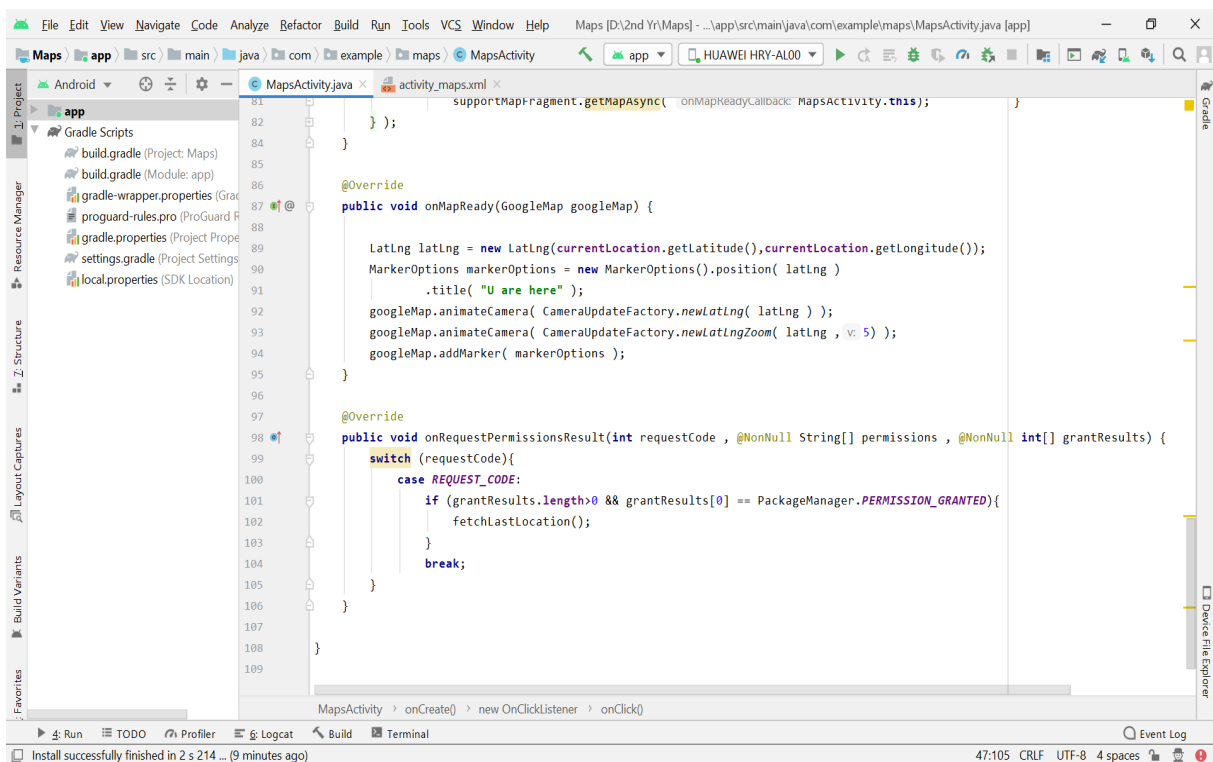


Figure 5.16: Getting Current Position

5.3 Camera Permissions

- We need to make a device manager so as to get device administrator permissions.
- Using device admin policies we can enable and disable camera.

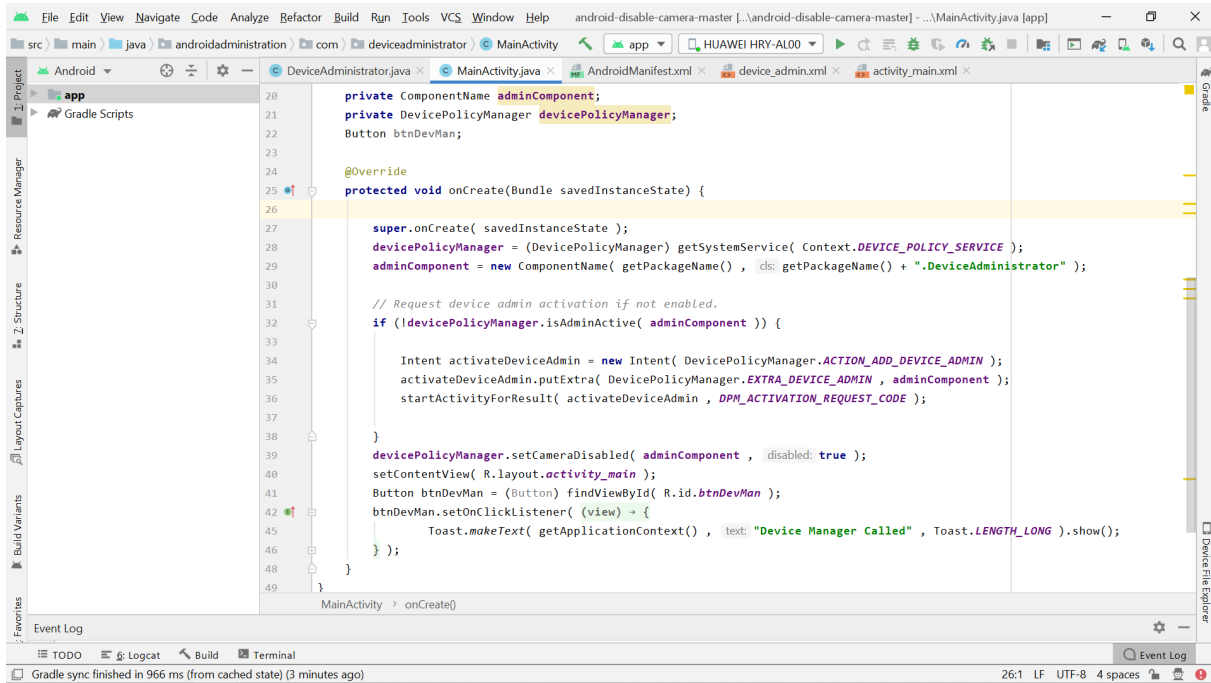


Figure 5.17: Main Activity

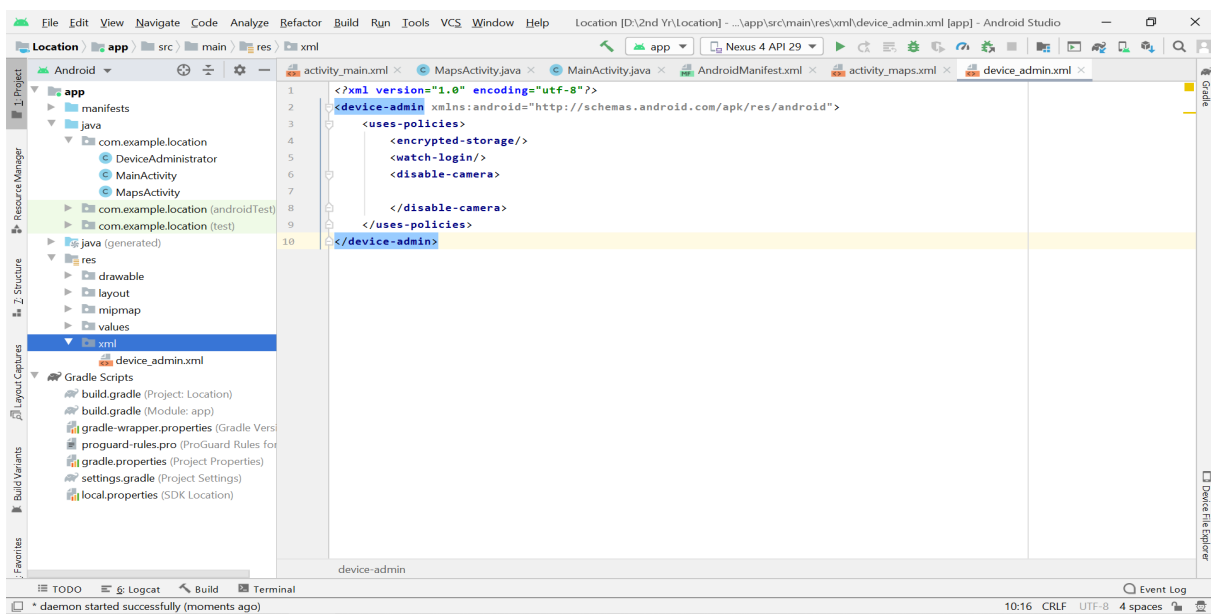


Figure 5.18: Defining Policies for Application

5.4 Database Security

- Libraries

The SQLite Android bindings code may also be added directly to the application project, so that it is built and deployed in the same way as all other application code. Add a dependency on the new module to the main application module (or to all modules that will use the SQLite Android bindings)

implementation 'androidx.sqlite:sqlite:2.1.0'

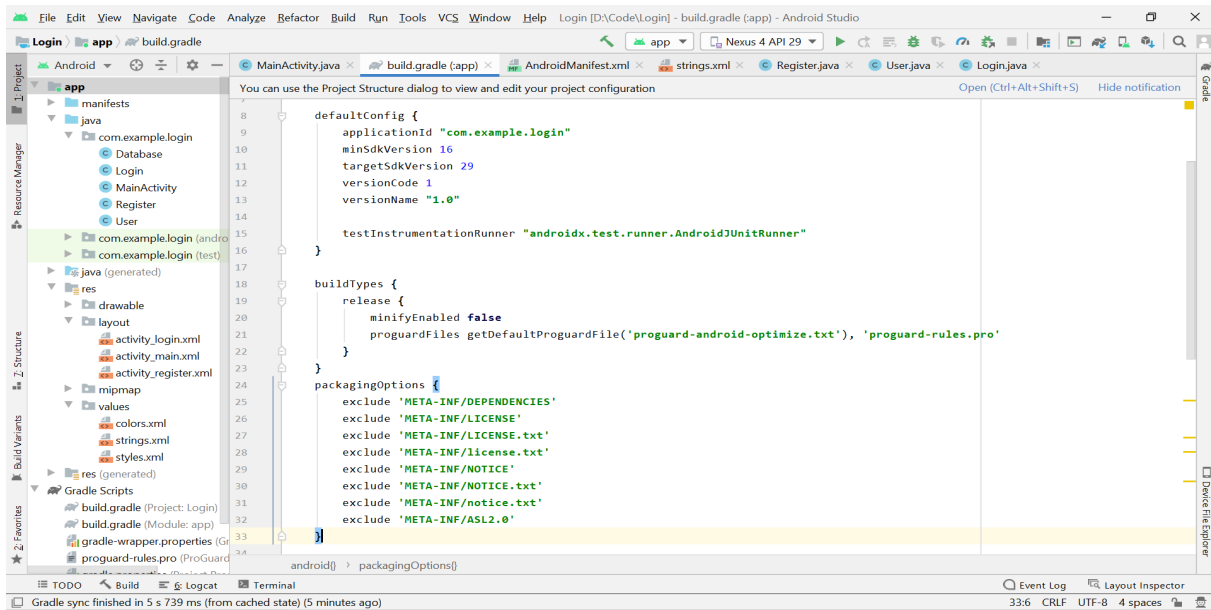


Figure 5.19: Defining Policies for Application

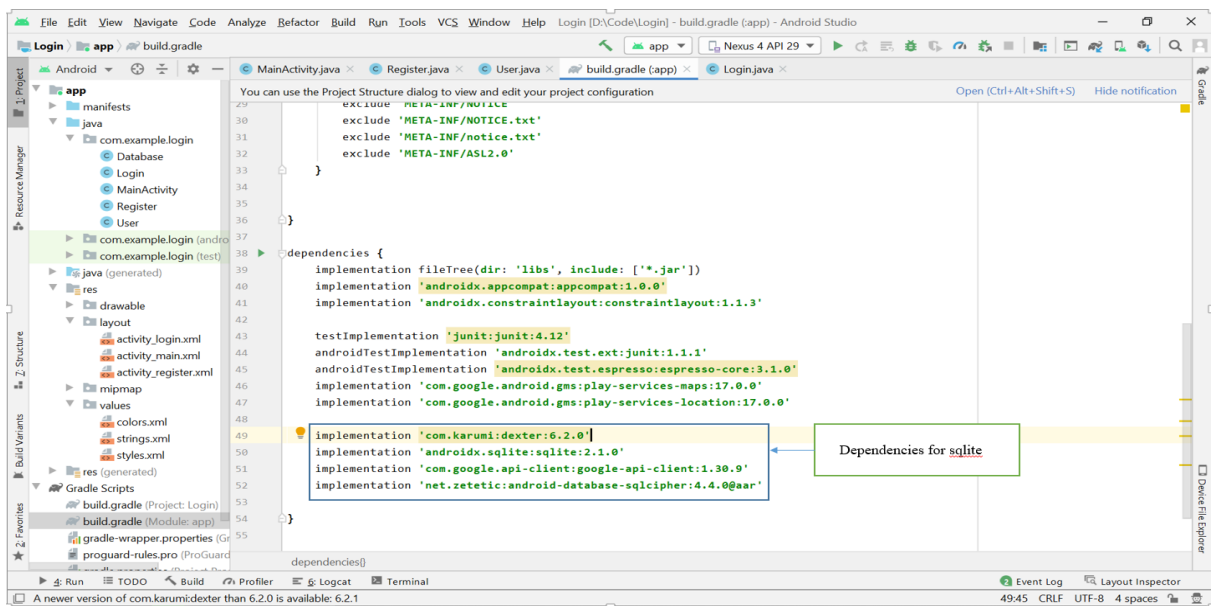


Figure 5.20: Defining Policies for Application

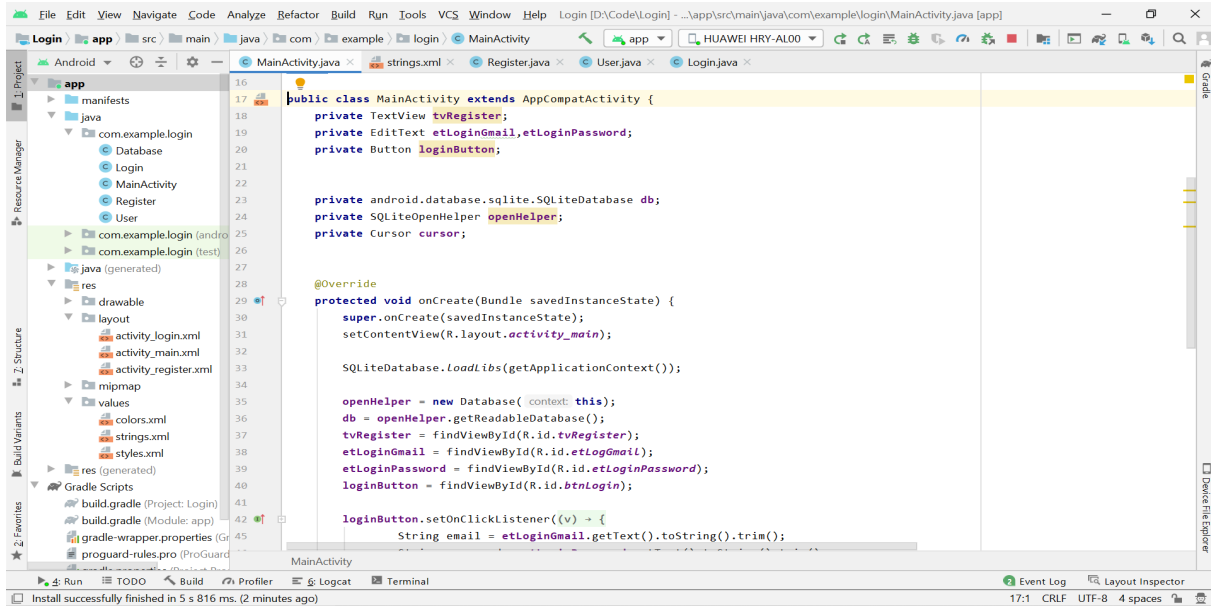


Figure 5.21: Defining Policies for Application

5.5 Main Code

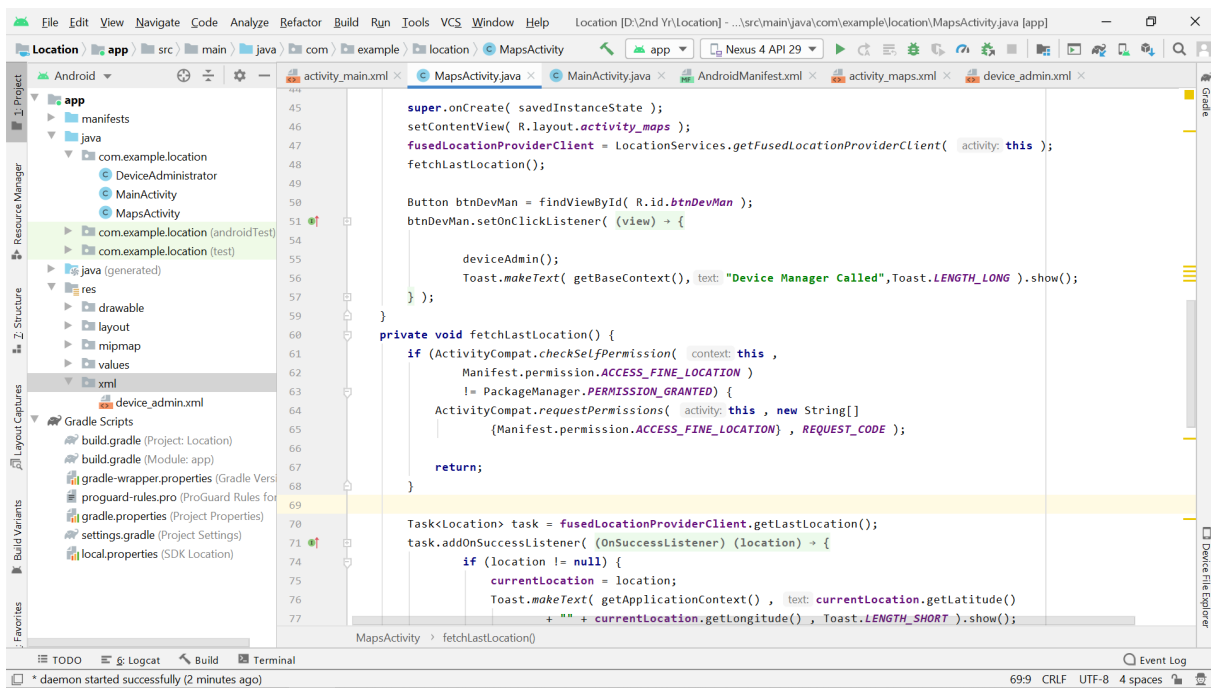


Figure 5.22: Code I

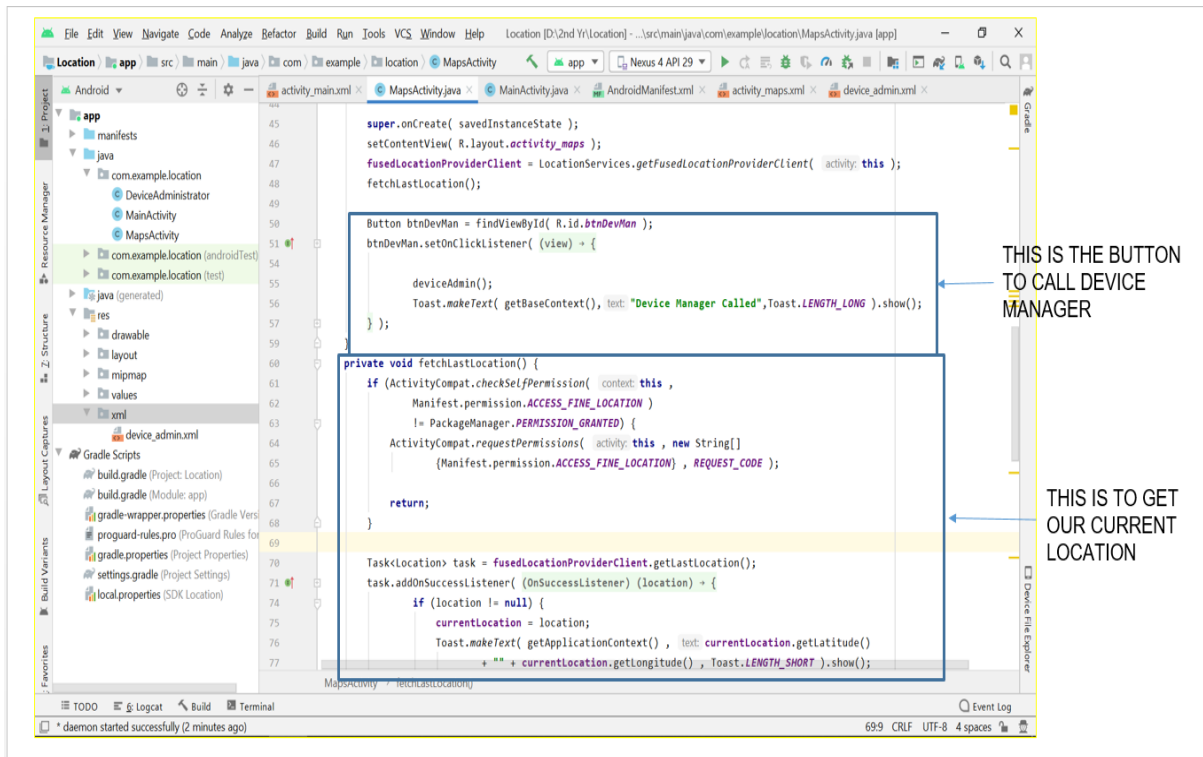


Figure 5.23: Code II

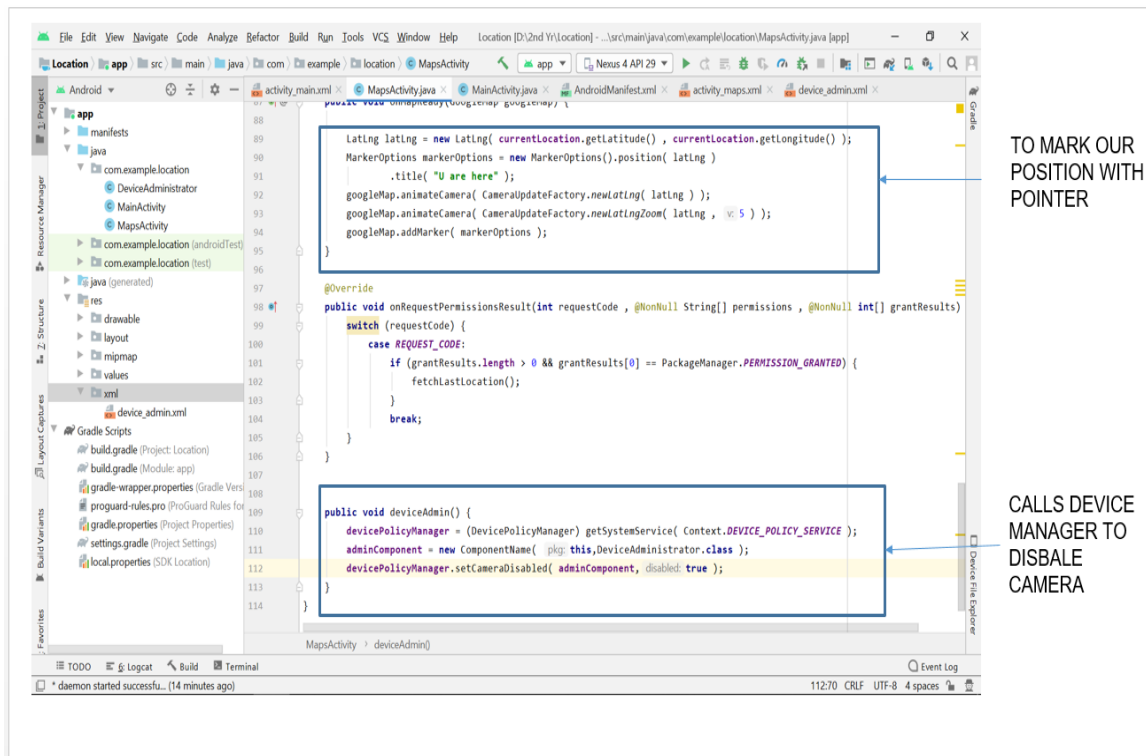


Figure 5.24: Code III

5.6 Implementation

The implementation section is divided into code implementation subsection and visualization subsection. In code implementation subsection the code snapshots for algorithms implemented are shown and explained. In visualization section the implementation results are presented in form of snapshots.

5.6.1 Code Implementation

1. Getting the users current location

The above snapshot shows the user's current location. The user's current location can be found by using Google Map's API. This key is generated once and can be used for the further procedure. Knowing a user's location in an Android app can be extremely useful. Users take their devices everywhere and are constantly using them on the go, and as developers, we can capitalise on that by providing a more contextual experience based on their current location. In this we are going to see how to get users current location using FusedLocationProviderApi. 1. It provides simple and easy to use APIs. 2. Provides high accuracy over other options. 3. Utilises low power by choosing the most efficient way to access the location.

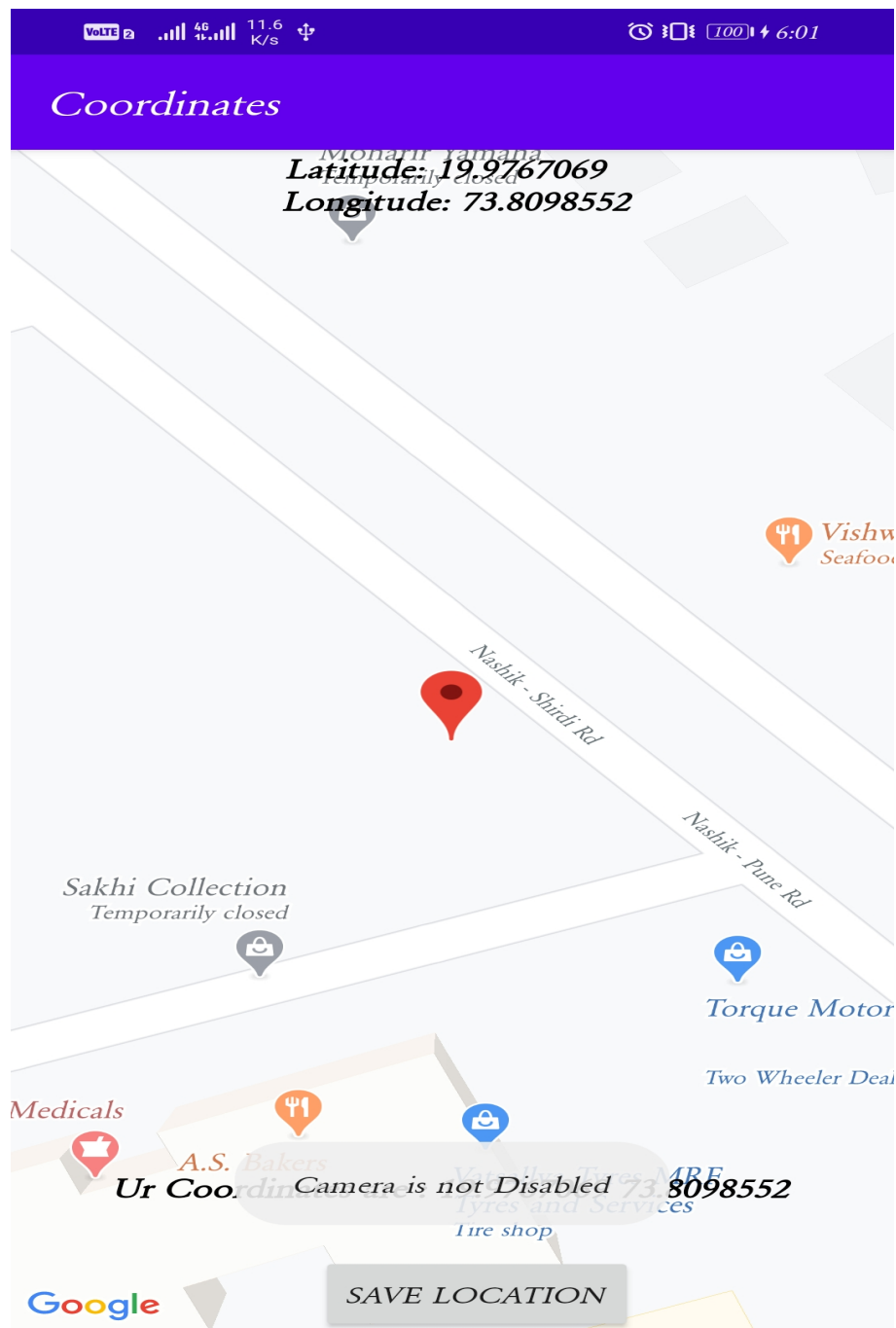


Figure 5.25: Getting Users Location



Figure 5.26: Device Admin Permission

2. Giving device admin access

The above snapshot shows the activity of Calling the device manager. Open the Android Device Manager to track and manage your devices.

3. Disabling Camera

The above snapshots shows the Disabled System Installed application. Android platform offers system-level device management capabilities through the Device Administration APIs. If an application package component is registered as system administrator, it can provide device administration features at the system level. An active system admin package can not be uninstalled until removed as system administrator. We will make benefits of this property to disable android application uninstall by registering and activating system admin component.

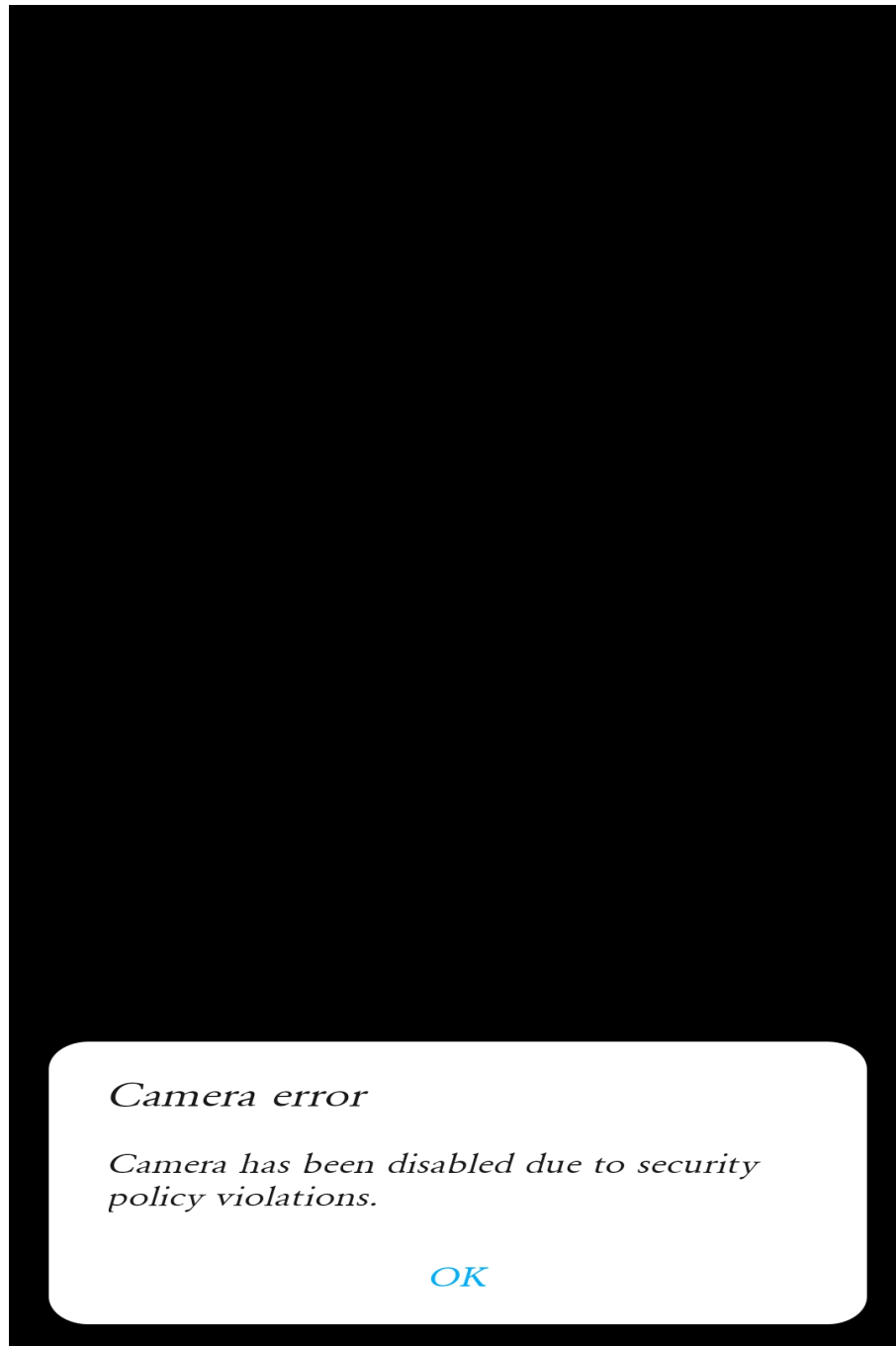


Figure 5.27: Camera Disabled

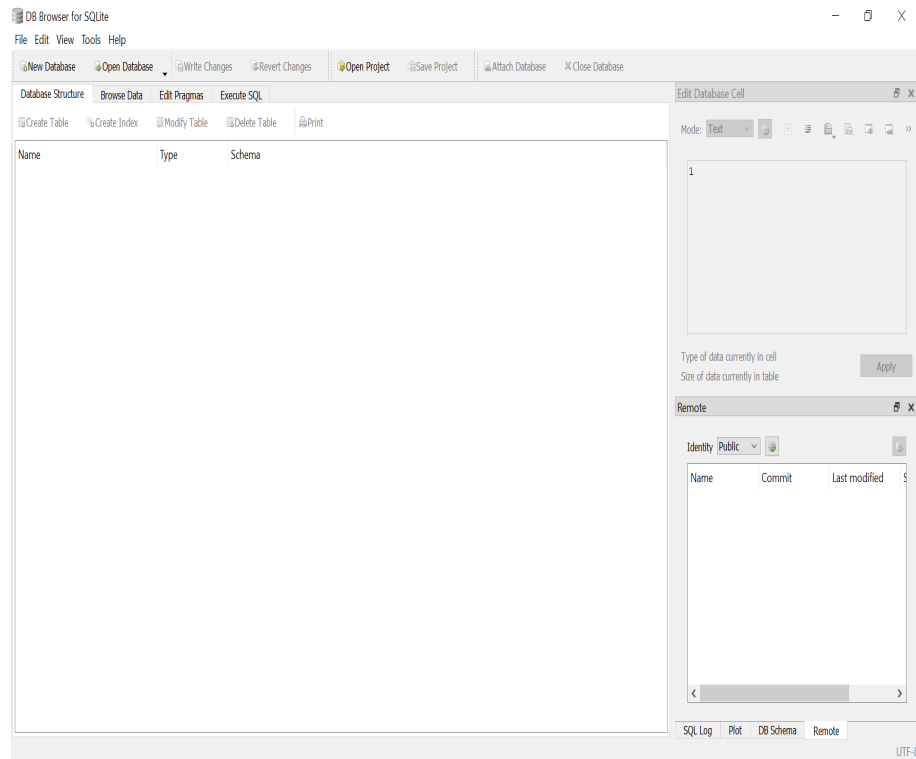


Figure 5.28: SqliteDatabaseBrowser Software

4. **For New Location , user needs to login**
5. **Register if new else login for old users**
6. **Give access to admin only for adding location**
7. **Save database file**
8. **Install SqliteDatabaseBrowser**
9. **Encrypt Database**

Chapter 6

Results

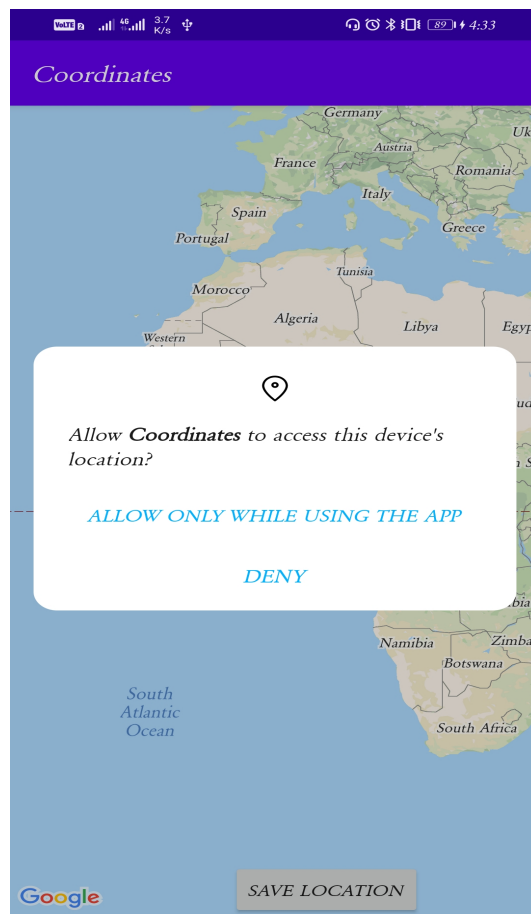


Figure 6.1: Getting Location Permissions

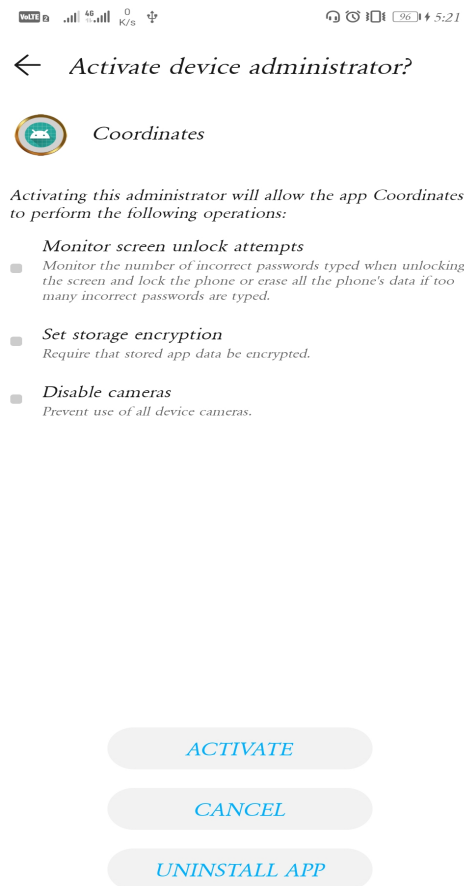


Figure 6.2: Device Admin Activation



Figure 6.3: Camera not disabled

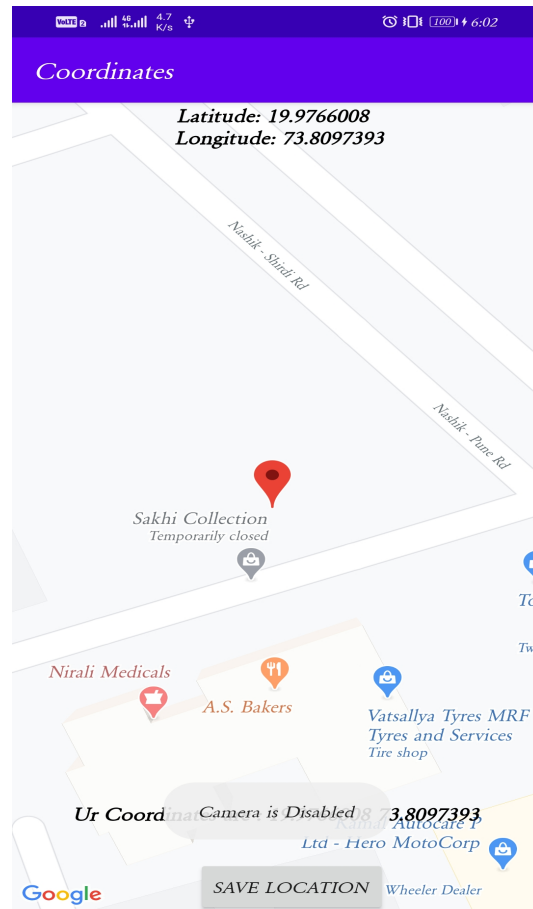


Figure 6.4: Camera Disabled

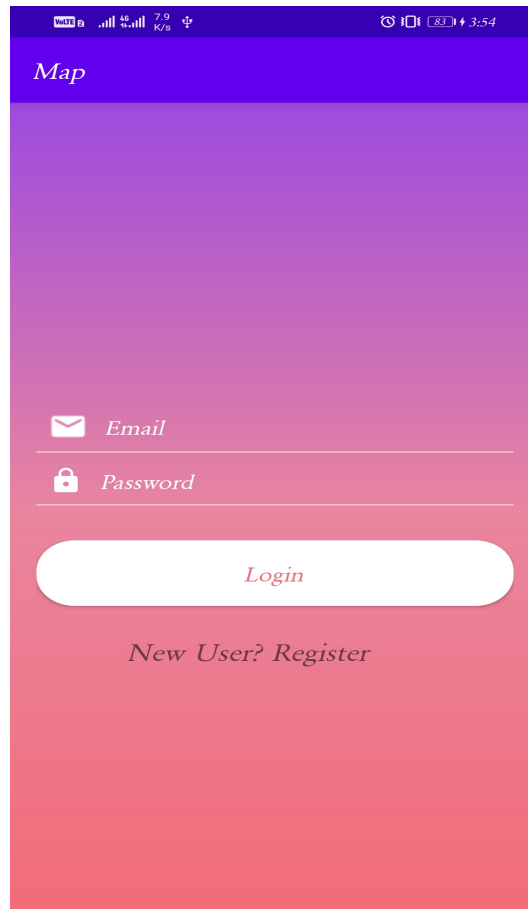


Figure 6.5: Registration Page for Admin

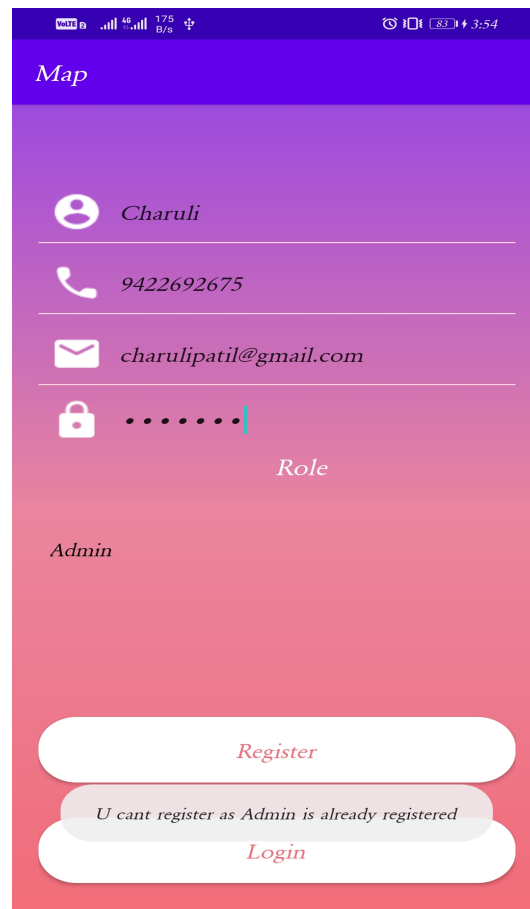


Figure 6.6: Registration Page for User

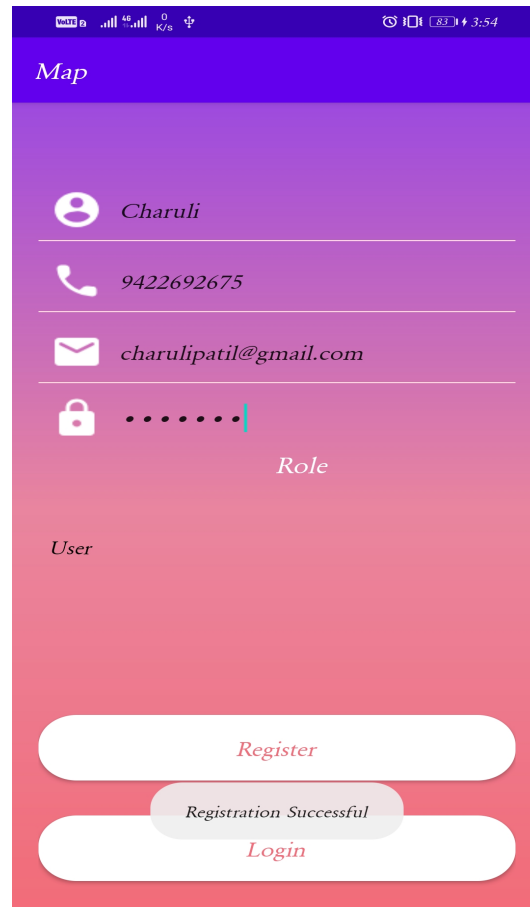


Figure 6.7: Code III

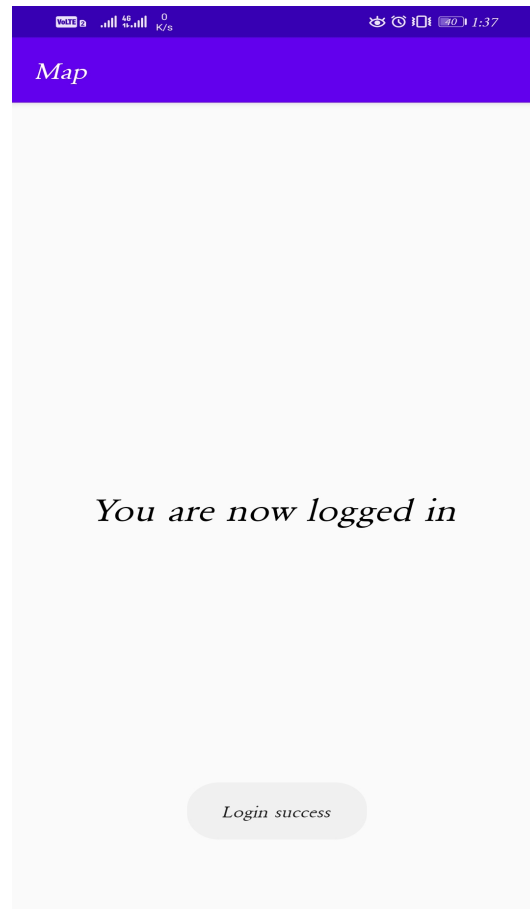


Figure 6.8: Code III

Chapter 7

Conclusion

7.1 Conclusion

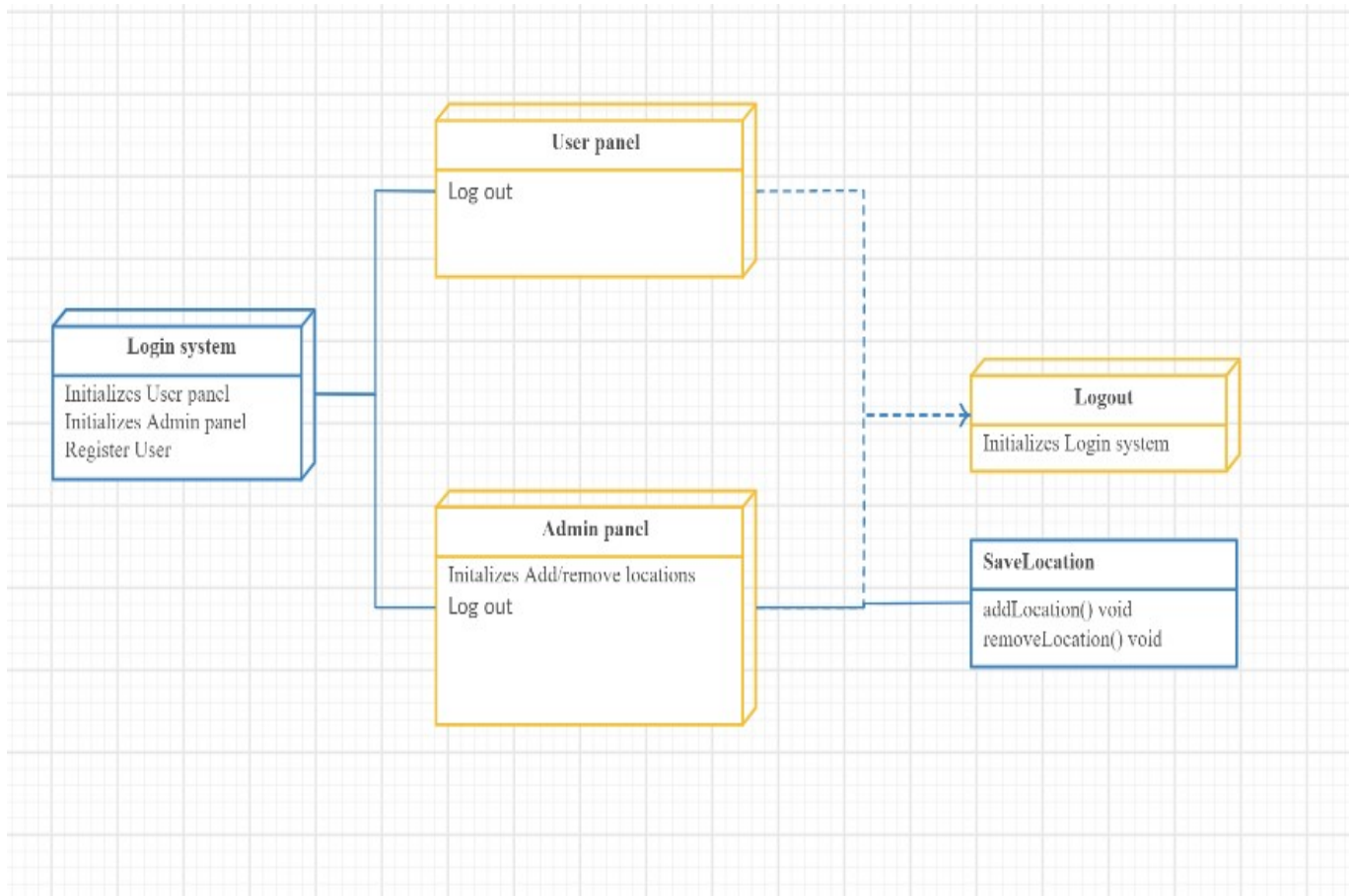
In this thesis, we proposed a new access control model for A.P.S. protecting user privacy. Our model allows for robust and retrospective approval based on predefined conditions. Our program is a variant of the CA-RBAC designed in such a way that contexts, have a set of Android permissions, are assigned requests and conditions are subject to permissions. This is different from the existing CA-RBAC models that match conditions and roles in accordance with our program that includes content and permissions. I believe that my proposed system can provide better privacy without any noticeable effect on the operation of the permit system. Our system can provide a valid and well-designed version of the Android version prior to the version of Android 7. We have also developed and implemented a demonstration system called CA-ARBAC. We performed various tests using our demonstration system and obtained the expected results. We will introduce the building of a secure data structure on Android. Our architecture will provide data privacy by allowing only the authorized business to access the associated database. Only sdbd can access a unique key in TEE and only an authorized application can request a purchase on sdbd. Therefore, for such a validation process, only the most patented applications can access that program.

7.2 Future Scope

Due to the various limitations we have, there are some constraints that we have to set for the future. The following is one of the future tasks we plan to accomplish in our next project. Active content management. In our current system, we use synthetic content management. In the current context management, once the request is granted, we can use it without any modification of the context information. However, in practical cases, the application must be restored to access if the context changes during the application. We were not able to use the current context management functionality in CA-ARBAC since the framework that uses the Android security policy does not support context management. The A.S.M. framework and does not support theme management. For the future, we propose a combination of security APIs such as A.S.M. supporting effective content management.

Appendices

A Deployment Diagram



7.2 Publications

Review Paper has been published in **International Research Journal of Engineering and Technology (IRJET) Volume 7, Issue 7, July 2020**
<https://irjet.net/volume7-issue7>, S.No: 176



e-ISSN: 2395-0056 p-ISSN: 2395-0072

International Research Journal of Engineering and Technology (IRJET)

(An ISO 9001 : 2008 Certified Journal)

We hereby awarding this certificate to

Prof. Kiran K Joshi

In recognition the publication of the manuscript entitled

***User Privacy and Database Security using Context based Access Control
in Android Devices: A Survey***

published in our Journal Volume 7 Issue 7 July 2020



Editor in Chief

Impact Factor : 7.529

www.irjet.net

E-mail : editor@irjet.net

Bibliography

- [1] Leveraging Access Control Mechanism of Android Smartphone Using Context-Related Role-Based Access Control Model *Thiri The' Wut Yee, Nilar Thein IEEE The 7th International Conference on Networked Computing and Advanced Information Management*
- [2] Does this App Really Need My Location? Context-Aware Privacy Management for Smartphones *Saksham Chitkara, Nishad Gothoskar, Suhas Harish, Jason I. Hong, and Yuvraj Agarwal, Proc. A.C.M. Interact Mob. Wearable Ubiquitous Technol. 1, 3, Article 42 (September 2017), 22 pages*
- [3] CA-ARBAC: privacy preserving using context-aware role-based access control on Android permission system *Tomur, Department of Computer Engineering, Izmir Institute of Technology, Urla, Izmir, Turkey, Published online 24 January 2017 in Wiley Online Library (wileyonlinelibrary.com)*
- [4] EasyPrivacy: Context-Aware Resource Usage Control System for Android Platform *Bingfei Ren1, Chuanchang Liu1, Bo Cheng 1, Shuangxi Hong 1, Jie Guo2 and Junliang Chen1, National Natural Science Foundation of China under Grant U1536112*
- [5] An SELinux-based Intent manager for Android *Simone Mutti, Enrico Bacis, Stefano Paraboschi, DIGIP — Universit'a degli Studi di Bergamo, Italy fsmone.mutti, enrico.bacis, paraboscg @ unibg.it*
- [6] Context-Based Access Control Systems for Mobile Devices *Bilal Shebaro, Oyindamola Oluwatimi, and Elisa Bertino, Fellow, IEEE, Article in IEEE Transactions on Dependable and Secure Computing · March 2015*
- [7] Security Architecture for a Secure Database on Android [8] *J. H. Park, S. Yoo, I. S. Kim and D. H. Lee, IEEE Access, vol. 6, pp. 11482-11501, 2018.*
- [8] CA-ARBAC: "privacy preserving using context-aware role-based access control on Android permission system *J. Abdella, M. zuysal and E. Tomur, Security and communication networks Security Comm. Networks (2017)*
- [9] Analysis On Preserving Location Privacy [4] *Rohan J. Patil, Prof. K.K. Joshi and Prof. Sowmiya Raksha, " International Journal Of Advance Research In Computer Science And Software Engineering, vol. 3, no. 3, pp. 562-566, March 2015.*